

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO ASSIGNMENT 06

**ĐỀ TÀI: MẠNG NƠ-RON HỒI QUY RNN, LSTM, GRU VÀ BiLSTM
TRÊN HAI TẬP DỮ LIỆU CHUỖI THỰC TẾ BẰNG PYTORCH VÀ KERAS**

Lớp: D23CTPM01

Họ và tên: Nguyễn Duy Nghĩa

Mã sinh viên: B23DCCN600

Giảng viên hướng dẫn: PGS.TS Trần Đình Quế

Học kỳ: Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 09/2026

Lời mở đầu

Các bài trước làm việc với dữ liệu mà thứ tự không quan trọng: một bệnh nhân, một căn nhà, một bức ảnh. Assignment 06 chuyển sang dữ liệu chuỗi, nơi thứ tự chính là thông tin. Giá cổ phiếu hôm nay phụ thuộc hôm qua; một người sắp hủy gói nghe nhạc thường nghe thưa dần trước khi dừng hẳn. Mạng nơ-ron hồi quy được tạo ra cho đúng loại dữ liệu này: nó đọc chuỗi từng bước và mang theo một trạng thái nhớ.

Bài dựng bốn kiến trúc hồi quy (Simple RNN, LSTM, GRU, BiLSTM) bằng cả PyTorch và Keras, huấn luyện trên hai tập dữ liệu thật, tổng cộng 16 mô hình. Tập thứ nhất là giá cổ phiếu Amazon trong 27 năm, một bài hồi quy. Tập thứ hai là log nghe nhạc theo ngày của 100.000 người dùng KKBox, một bài phân loại người dùng rồi bỏ dịch vụ. Chương 1 dựng lý thuyết từ công thức, mỗi khái niệm là một hàm NumPy đối chiếu với PyTorch. Chương 2 phân tích và tiền xử lý hai tập. Chương 3 và 4 huấn luyện 16 mô hình rồi so sánh hai framework. Chương 5 đóng gói các mô hình thành một Web App.

Hai tập cho hai kết quả trái ngược, và cả hai đều có ích. Trên KKBox, các mô hình học được một tín hiệu thật: người sắp rời bỏ nghe nhiều hơn ở đầu tháng rồi giảm dần, và LSTM đạt ROC-AUC 0,6819 chỉ từ chuỗi hành vi. Trên AMZN, không mô hình nào trong 16 mô hình thắng được mốc ngây thơ “giá ngày mai bằng giá hôm nay” (RMSE 3,12 USD). Báo cáo trình bày kết quả này như nó là, kèm phân tích vì sao, thay vì dựa vào chỉ số R^2 xấp xỉ 0,99 trên giá, một chỉ số trông rất đẹp nhưng mốc ngây thơ cũng đạt được.

Mọi con số trong báo cáo được sinh tự động từ tệp kết quả của notebook, mọi đoạn mã được trích thẳng từ mã nguồn, và mọi hình được vẽ bằng TikZ từ dữ liệu, kể cả hai khung giao diện của Web App. Chạy lại notebook thì báo cáo đổi theo. Phụ lục A ghi các lệnh để tái lập.

Em xin cảm ơn thầy PGS.TS Trần Đình Quế đã hướng dẫn học phần.

Hà Nội, tháng 9 năm 2026

Nguyễn Duy Nghĩa

Đối chiếu với các hạng mục của bài

Hạng mục	Chương	Notebook	Mã nguồn
1. Cơ sở lý thuyết RNN, kèm code	1	00_ly_thuyet_rnn	a06/theory.py
2. Phân tích và tiền xử lý hai tập dữ liệu chuỗi thực tế	2	01_du_lieu_amzn, 02_du_lieu_kkbox	a06/data_stock.py, a06/data_churn.py
3. Hiện thực và huấn luyện bằng PyTorch	3	03_pytorch_amzn, 04_pytorch_kkbox	a06/models_torch.py, a06/train_torch.py
4. Hiện thực Keras, đối chuẩn PyTorch và Keras	4	05_keras, 06_doi_chuan	a06/models_keras.py, a06/bench.py
5. Đóng gói triển khai Web App	5	—	app/
6. Kết luận	6	—	—

Mục lục

Lời mở đầu	i
Mục lục	i
Danh mục Hình vẽ	iv
Danh mục Bảng biểu	v
Danh mục Mã nguồn	vi
1 Cơ sở lý thuyết mạng nơ-ron hồi quy	1
1.1 Dữ liệu chuỗi và tensor ba chiều	1
1.2 Các dạng bài toán chuỗi	1
1.3 Vì sao MLP và Conv1D chưa đủ	2
1.4 Simple RNN	4
1.5 Lan truyền ngược theo thời gian và hai vấn đề của gradient	4
1.6 Gradient clipping	6
1.7 LSTM	7
1.8 GRU	8
1.9 LSTM hai chiều	9
1.10 Điều chuẩn cho mạng hồi quy	10
2 Hai tập dữ liệu chuỗi thực tế	12
2.1 Giá cổ phiếu AMZN	12
2.1.1 Nguồn và quy mô	12
2.1.2 Return, phân phối và biến động	13
2.1.3 Tính dừng: kiểm định ADF và tự tương quan	14
2.1.4 Cửa sổ trượt và chống rò rỉ	15
2.2 KKBBox Churn	16
2.2.1 Nguồn và định nghĩa nhân	16
2.2.2 Lấy mẫu và dựng chuỗi	17
2.2.3 Phân tích dữ liệu	18
3 Hiện thực và huấn luyện bằng PyTorch	21
3.1 Một lớp cho bốn mô hình	21
3.2 Vòng lặp huấn luyện	21
3.2.1 Chọn thiết bị bằng đo	23
3.3 Kết quả trên AMZN	23
3.4 Kết quả trên KKBBox	25
4 Keras và đối chuẩn hai framework	27
4.1 Bốn mô hình bằng Keras	27
4.1.1 Chênh lệch số tham số	28
4.2 Huấn luyện song song trên hai thiết bị	28
4.3 Đo độ trễ suy luận	29

4.4	Bảng đối chuẩn 16 mô hình	30
4.5	Giới hạn của mô hình dự báo giá	31
5	Triển khai Web App	32
5.1	Kiến trúc	32
5.2	Nạp mô hình	33
5.3	Hai phân hệ dự báo	34
5.4	Thiết kế giao diện	35
5.5	Giao diện phân hệ 1: dự báo giá cổ phiếu AMZN	37
5.6	Giao diện phân hệ 2: đánh giá nguy cơ rời bỏ KKBox	38
5.6.1	Trình mô phỏng hành vi	39
5.7	Kiểm chứng	41
6	Kết luận	42
6.1	Trả lời sáu hạng mục	42
6.2	Hạn chế	42
6.3	Hướng phát triển	42
	Tài liệu tham khảo	44
A	Tái lập kết quả	46
A.1	Môi trường	46
A.2	Các bước	46

Danh mục Hình vẽ

1.1	Tensor dữ liệu chuỗi; lát tô cam là một bước thời gian x_t	1
1.2	Năm dạng bài toán chuỗi; ô xanh lá là đầu vào, xanh dương là trạng thái ẩn, cam là đầu ra	2
1.3	RNN ở dạng cuộn và dạng mở theo thời gian	4
1.4	Tích Jacobi co hoặc giãn theo hàm mũ tùy bán kính phổ của W_{hh}	5
1.5	Gradient của mất mát theo h_t , chuẩn hoá theo bước cuối (trục tung thang log)	6
1.6	Gradient clipping chỉ đổi độ dài của g , không đổi hướng	7
1.7	Tế bào LSTM: đường cộng của cell state ở trên, bốn phép biến đổi có cổng ở dưới	8
1.8	Tế bào GRU với hai cổng	8
1.9	BiLSTM: vector tóm tắt là trạng thái cuối của hai chiều đọc	9
1.10	Mặt nạ dropout thường và recurrent dropout; ô đỏ là đơn vị bị tắt	10
2.1	Giá đóng cửa AMZN (thang log) với MA20, MA50 và khối lượng; vạch đứt là ranh giới train, val, test	13
2.2	Log-return, phân phối return so với phân phối chuẩn, biến động 20 phiên và tương quan giữa 8 đặc trưng	14
2.3	ACF và PACF 40 trễ của giá và của return; dải xám là khoảng tin cậy 95 %	15
2.4	Cửa sổ trượt và cách chia theo thời gian của AMZN	16
2.5	Phân bố nhãn trong mẫu và tỉ lệ churn qua các bước lọc	18
2.6	Quỹ đạo trung bình 31 ngày của nhóm ở lại và nhóm churn	19
2.7	Trái: phân phối tổng giờ nghe trong tháng. Phải: tương quan giữa 8 đặc trưng	19
2.8	Tương quan từng đặc trưng với nhãn churn theo ba cách tóm tắt chuỗi 31 ngày	20
3.1	Đường loss của bốn mô hình PyTorch trên AMZN (liền: train, đứt: val)	23
3.2	Giá thật, mốc ngày thơ và dự báo của bốn mô hình trên các phiên cuối của tập test	24
3.3	Đường loss của bốn mô hình PyTorch trên KKBox (liền: train, đứt: val)	25
3.4	Ma trận nhầm lẫn trên test KKBox; màu ô theo tỉ lệ trong hàng	26
3.5	Đường ROC và Precision–Recall trên test KKBox; vạch đứt là mô hình đoán ngẫu nhiên	26
4.1	Số tham số và kích thước tệp mô hình của hai framework	28
4.2	Đường loss của bốn mô hình Keras trên AMZN (liền: train, đứt: val)	29
4.3	Đường loss của bốn mô hình Keras trên KKBox (liền: train, đứt: val)	29
4.4	Chất lượng theo độ trễ suy luận của 16 mô hình	31
5.1	Kiến trúc Web App	32
5.2	Phần đầu trang: bốn ô số liệu và hai nút chuyển phân hệ	36
5.3	Phân hệ 1 ở nền sáng	37
5.4	Phân hệ 1: dự báo giá AMZN phiên kế tiếp bằng mô hình PyTorch tốt nhất trên validation, kèm bảng so sánh bốn kiến trúc	38
5.5	Phân hệ 2: đánh giá nguy cơ rời bỏ của một người dùng thật đã churn trong tập test	39
5.6	Trình mô phỏng: nghe 90 phút, 25 bài mỗi ngày, 5 ngày một tuần, ngừng nghe từ ngày 12	40

Danh mục Bảng biểu

- 1.1 Tầng đầu của ba loại mô hình cho chuỗi $T = 30, D = 8, H = 64$ 2
- 1.2 So sánh ba loại mô hình trên dữ liệu chuỗi 3
- 1.3 So sánh Simple RNN, LSTM, GRU và BiLSTM ($D = 8, H = 64$, tham số của riêng tầng hồi quy trong PyTorch) 9

- 2.1 Thống kê mô tả 10 đặc trưng của AMZN, 6.836 phiên 12
- 2.2 Kiểm định t Welch giữa nhóm ở lại và nhóm churn, trung bình 31 ngày của từng người 20

- 3.1 Cấu hình huấn luyện 22
- 3.2 Bốn mô hình PyTorch trên tập test AMZN (2020-09-01 đến 2024-09-24) 24
- 3.3 Bốn mô hình PyTorch trên tập test KKBox (15.000 người dùng) 25

- 4.1 Đối chuẩn trên AMZN: 8 mô hình và mốc ngày thơ 30
- 4.2 Đối chuẩn trên KKBox: 8 mô hình 30

- 5.1 REST API của Web App 33

Danh mục Mã nguồn

1	Trường thụ cảm của Conv1D thường và giãn nở	3
2	Tích chập nhân quả: đệm lệch trái để không nhìn thấy tương lai	3
3	Số tham số của Simple RNN không phụ thuộc độ dài chuỗi	3
4	Lan truyền xuôi của Simple RNN theo công thức (1.2)	4
5	Chuẩn của tích Jacobi với bán kính phổ ρ cho trước	5
6	Đo chuẩn gradient theo từng bước bằng autograd	6
7	Cắt gradient theo chuẩn, giữ hướng	7
8	Một bước LSTM theo (1.4), thứ tự khối công như PyTorch	8
9	Một bước GRU theo (1.5)	9
10	Hai kiểu mặt nạ dropout cho chuỗi	10
11	Layer Normalization theo từng mẫu	10
12	Thêm đặc trưng tài chính từ giá và khối lượng	12
13	Cửa sổ trượt với nhãn là return phiên kế tiếp	15
14	Chuẩn hoá đặc trưng và nhãn, fit chỉ trên train	16
15	Lấy mẫu phân tầng những người vừa có nhãn vừa có log	17
16	Gom log theo ngày thành tensor $(N, 31, 8)$	18
17	Chia train/val/test phân tầng theo nhãn	20
18	Lớp chung cho Simple RNN, LSTM, GRU và BiLSTM	21
19	Vòng lặp huấn luyện chung: AdamW, clip gradient, dừng sớm theo val loss	22
20	Hàm mất mát cho hai loại bài toán	23
21	Chọn CPU hay GPU bằng cách đo một epoch	23
22	Chọn ngưỡng theo F1 trên validation	25
23	Bốn mô hình Keras dựng từ một hàm	27
24	Trọng số mẫu thay cho pos_weight	27
25	Huấn luyện một mô hình Keras với cùng lô và cùng dừng sớm	28
26	Đo thời gian: khởi động rồi lấy trung vị	29
27	Độ trễ với lô 1 và thông lượng với lô 256	30
28	Nạp mô hình một lần, cùng giao diện cho PyTorch và Keras	33
29	Endpoint dự báo giá phiên kế tiếp	34
30	Endpoint dự báo churn	35
31	Xu hướng nghe trả về cùng xác suất churn	35
32	Thẻ mô hình: chỉ số test, số tham số, độ trễ và kích thước tệp	36
33	Dựng chuỗi 31 ngày giả định từ bốn thanh trượt	40
34	Kiểm chứng bốn endpoint và ghi phản hồi thật cho báo cáo	41
35	Gom dữ liệu KKBox một lần và lưu lại	46

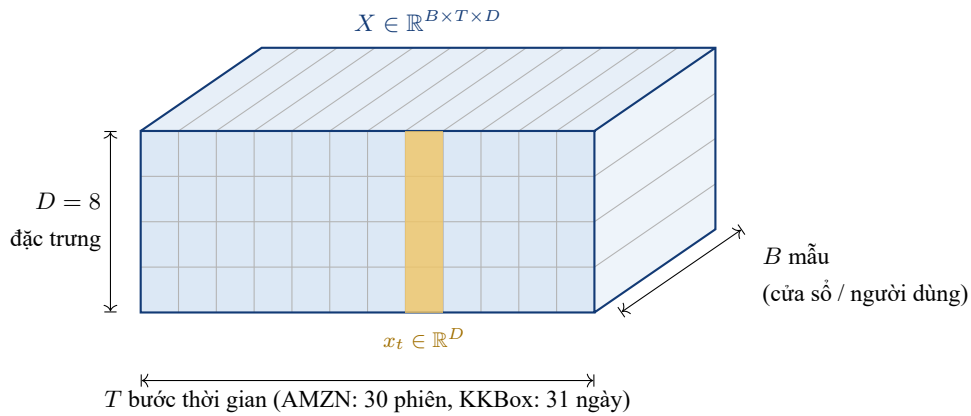
Chương 1

Cơ sở lý thuyết mạng nơ-ron hồi quy

Chương này dựng lại các khái niệm của mạng hồi quy từ công thức. Mỗi khái niệm là một hàm NumPy trong `a06/theory.py`; notebook 00 chạy hàm đó và so với lớp PyTorch tương ứng bằng `np.allclose`. Cả 11 phép đối chiếu đều cho kết quả khớp, nên các công thức dưới đây chính là những gì thư viện đang tính.

1.1 Dữ liệu chuỗi và tensor ba chiều

Một mẫu dữ liệu chuỗi là một dãy vector $x_1, x_2, \dots, x_T$ với $x_t \in \mathbb{R}^D$. Gom B mẫu lại được tensor $X \in \mathbb{R}^{B \times T \times D}$ (Hình 1.1). Cả hai tập của bài đều có dạng này. AMZN cho X_{train} có shape $(4.764, 30, 8)$: mỗi mẫu là một cửa sổ 30 phiên giao dịch, mỗi phiên 8 đặc trưng. KKBox cho $(70.000, 31, 8)$: mỗi mẫu là một người dùng, 31 ngày của tháng 3/2017, mỗi ngày 8 đặc trưng nghe nhạc.



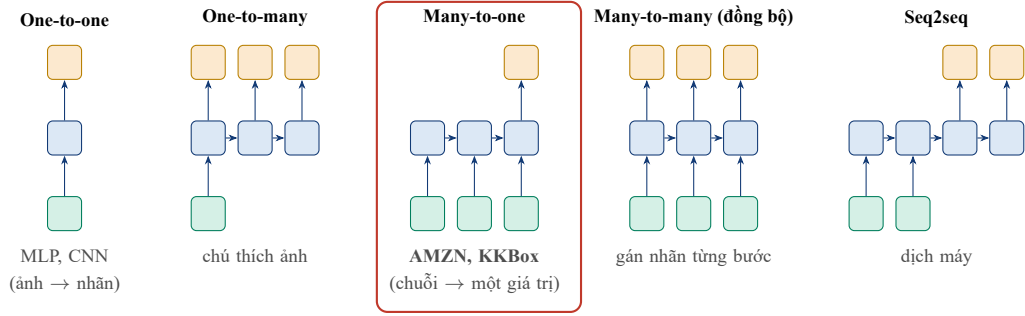
Hình 1.1. Tensor dữ liệu chuỗi; lát tô cam là một bước thời gian x_t

Điều làm dữ liệu chuỗi khác dữ liệu bảng là thứ tự. Giá hôm nay phụ thuộc giá hôm qua; một người sắp bỏ dịch vụ thường nghe ít dần trong những ngày cuối. Giả định các mẫu độc lập và cùng phân phối (i.i.d.) vẫn đúng giữa các mẫu, nhưng sai hoàn toàn giữa các bước trong cùng một mẫu. Mô hình cho chuỗi phải khai thác được sự phụ thuộc đó.

Cùng một tensor có hai cách xếp trục. Thư viện cuDNN của NVIDIA và mặc định của `nn.LSTM` dùng bố cục *time-first* (T, B, D) : mỗi bước thời gian là một khối liền trong bộ nhớ, thuận cho vòng lặp theo t . Keras và phần lớn mã PyTorch dùng *batch-first* (B, T, D) với `batch_first=True`, cùng bố cục với các tầng Dense và tích chập. Đây chỉ là hoán vị trục: notebook 00 nạp cùng trọng số vào hai lớp LSTM, đưa vào tensor $(4, 30, 8)$ và bán hoán vị $(30, 4, 8)$, và nhận cùng đầu ra (đúng). Cả bài dùng *batch-first*.

1.2 Các dạng bài toán chuỗi

Theo quan hệ giữa chuỗi vào và chuỗi ra, bài toán chuỗi chia thành năm dạng (Hình 1.2). One-to-one là mạng thường của các bài trước: một đầu vào cố định, một đầu ra. One-to-many sinh một chuỗi từ một đầu vào, như viết chú thích cho ảnh. Many-to-one đọc cả chuỗi rồi cho một kết quả. Many-to-many đồng bộ cho một đầu ra ở mỗi bước, như gán nhãn từ loại. Seq2seq (bộ mã hoá–giải mã) đọc hết chuỗi vào rồi sinh chuỗi ra có độ dài khác, như dịch máy.



Hình 1.2. Năm dạng bài toán chuỗi; ô xanh lá là đầu vào, xanh dương là trạng thái ẩn, cam là đầu ra

Cả hai bài của báo cáo đều là **many-to-one**. AMZN đọc 30 phiên và cho một số là return của phiên kế tiếp. KKBox đọc 31 ngày và cho một xác suất rời bỏ. Vì vậy mọi mô hình ở các chương sau chỉ dùng trạng thái cuối h_T và bỏ qua $h_1, \dots, h_{T-1}$.

1.3 Vì sao MLP và Conv1D chưa đủ

Có hai cách dùng lại công cụ của các bài trước. Cách thứ nhất là làm phẳng chuỗi thành vector $T \cdot D$ chiều rồi đưa vào MLP. Cách thứ hai là trượt một kernel Conv1D theo trục thời gian. Bảng 1.1 đếm tham số của tầng đầu tiên với $T = 30$, $D = 8$ và $H = 64$ đơn vị ẩn.

Bảng 1.1. Tầng đầu của ba loại mô hình cho chuỗi $T = 30$, $D = 8$, $H = 64$

Mô hình	Số tham số	Giá trị	Độ dài chuỗi thay đổi	Nhớ quá khứ xa
MLP (làm phẳng)	$(T \cdot D)H + H$	15.424	không, T cố định	mọi bước, nhưng không chia sẻ trọng số
Conv1D, $K = 3$	$KDH + H$	1.600	được	chỉ K bước mỗi tầng
Simple RNN	$H(D + H + 1)$	4.672	được	qua trạng thái h_t

MLP học một trọng số riêng cho từng cặp (bước, đặc trưng). Nhân đôi độ dài chuỗi thì số tham số tầng đầu tăng từ 15.424 lên 30.784. Điều học được ở bước 5 cũng không dùng lại được ở bước 25: một mẫu hình giá xuất hiện ở hai vị trí khác nhau phải được học hai lần, vì không có chia sẻ trọng số theo thời gian.

Conv1D chia sẻ trọng số theo thời gian và tính song song được trên mọi vị trí, nên là lựa chọn mạnh cho nhiều bài chuỗi (WaveNet, TCN). Nhưng nó có ba giới hạn so với RNN:

1. *Trường thụ cảm hữu hạn.* Mỗi tầng kernel K chỉ nhìn tầm nhìn thêm $K - 1$ bước:

$$RF_{\text{thường}} = 1 + L(K - 1), \quad RF_{\text{giãn nở}} = 1 + (K - 1)(2^L - 1). \quad (1.1)$$

Với $K = 3$, muốn nhìn đủ $T = 30$ bước phải chồng 15 tầng; tích chập giãn nở của TCN rút xuống 4 tầng. Dù cách nào, tầm nhìn vẫn là hằng số chốt lúc thiết kế, còn h_t của RNN về nguyên lý mang thông tin từ bước đầu tiên.

2. *Tính nhân quả phải tự đảm bảo.* Conv1D với padding='same' đặt kernel đối xứng quanh t , nên đầu ra ở bước t dùng cả $x_{t+1}, \dots, x_{t+K/2}$: mô hình nhìn thấy tương lai. Notebook 00 đo trực tiếp bằng autograd: chuẩn gradient của đầu ra bước t theo các đầu vào sau t là 2 với padding='same', bằng 0 với tích chập nhân quả (đếm $K - 1$ giá trị bên trái) và bằng 0 với RNN. RNN nhân quả sẵn, không cần cấu hình gì.
3. *Không có trạng thái.* Khi một điểm dữ liệu mới x_{T+1} đến, RNN chỉ cần một bước $h_{T+1} = f(h_T, x_{T+1})$, chi phí $O(1)$. Conv1D phải tính lại trên cả cửa sổ.

```

1 # a06/theory.py
2 def receptive_field(L, K, dilated=False):
3     """Số bước quá khứ một nơ-ron ở tầng L của Conv1D nhìn thấy.
4
5     Thường:  $1 + L(K-1)$ , tăng tuyến tính theo số tầng. Giãn nở  $2^L$  (TCN):  $1 + (K-1)(2^L - 1)$ .
6     Dù Loại nào cũng là một hằng số chốt lúc thiết kế; RNN không có giới hạn này.
7     """
8     return 1 + (K - 1) * (2 ** L - 1) if dilated else 1 + L * (K - 1)

```

Mã nguồn 1. Trường thụ cảm của Conv1D thường và giãn nở

```

1 # a06/theory.py
2 class CausalConv1d(nn.Module):
3     """Tích chập nhân quả: đệm K-1 giá trị bên TRÁI, không đệm bên phải, nên y_t chỉ dùng x_{t-K+1..t}."""
4
5     def __init__(self, c_in, c_out, K):
6         super().__init__()
7         self.pad = K - 1
8         self.in_channels = c_in
9         self.conv = nn.Conv1d(c_in, c_out, K)
10
11     def forward(self, x):
12         return self.conv(nn.functional.pad(x, (self.pad, 0)))

```

Mã nguồn 2. Tích chập nhân quả: đệm lệch trái để không nhìn thấy tương lai

Đổi lại, RNN phải tính tuần tự theo t , nên huấn luyện khó song song hoá hơn Conv1D. Bảng 1.2 tổng hợp các tiêu chí.

Bảng 1.2. So sánh ba loại mô hình trên dữ liệu chuỗi

Tiêu chí	MLP (làm phẳng)	Conv1D	RNN, LSTM, GRU
Chia sẻ trọng số	không	cục bộ qua kernel	toàn cục qua mọi bước
Tầm nhìn quá khứ	cả cửa sổ, T cố định	$1 + L(K - 1)$ bước	không giới hạn về nguyên lý
Độ dài chuỗi thay đổi	không	được, qua pooling	được, tự nhiên
Tính nhân quả	phụ thuộc cách làm phẳng	cần tích chập nhân quả	sẵn có
Trạng thái nhớ	không	không	h_t (và c_t với LSTM)
Thêm một điểm mới	tính lại cả cửa sổ	tính lại cả cửa sổ	một bước, $O(1)$
Song song hoá	hoàn toàn	hoàn toàn	tuần tự theo t

Ba hàm đếm tham số nằm trong `theory.py`; notebook so từng công thức với `numel()` của `nn.Linear`, `nn.Conv1d` và `nn.RNN`.

```

1 # a06/theory.py
2 def count_params_rnn(D, H):
3     """Simple RNN:  $W_{xh}$  ( $H \times D$ ),  $W_{hh}$  ( $H \times H$ ),  $b$  ( $H$ ), không phụ thuộc  $T$ ."""
4     return H * (D + H + 1)

```

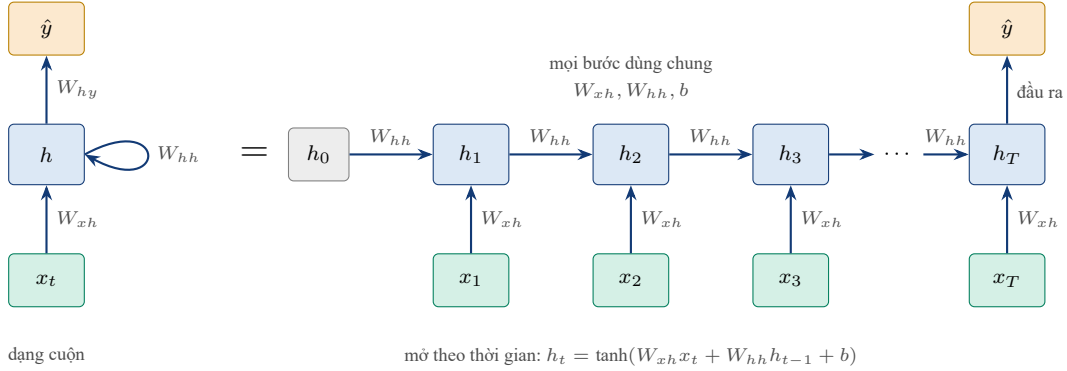
Mã nguồn 3. Số tham số của Simple RNN không phụ thuộc độ dài chuỗi

1.4 Simple RNN

Elman [1] đưa ra dạng đơn giản nhất: ở mỗi bước, trạng thái mới là hàm của đầu vào hiện tại và trạng thái cũ,

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b), \quad h_0 = 0, \quad \hat{y} = g(W_{hy}h_T + b_y). \quad (1.2)$$

Ở đây $W_{xh} \in \mathbb{R}^{H \times D}$, $W_{hh} \in \mathbb{R}^{H \times H}$, còn g là hàm đồng nhất với bài hồi quy và sigmoid với bài phân loại nhị phân. Vì bài là many-to-one, đầu ra chỉ tính từ h_T . Mỗi vòng lặp theo thời gian thì RNN trở thành một mạng sâu T tầng, nhưng mọi tầng dùng chung W_{xh} , W_{hh} và b (Hình 1.3). Trạng thái cuối h_T là bản tóm tắt cả chuỗi, và cả bốn mô hình của bài đều đưa h_T vào một đầu dự báo nhỏ.



Hình 1.3. RNN ở dạng cuộn và dạng mở theo thời gian

```
1 # a06/theory.py
2 def rnn_forward(X, W_xh, W_hh, b, h0=None):
3     """h_t = tanh(W_xh x_t + W_hh h_{t-1} + b). X: (T, D) -> (T, H), cùng W ở mọi bước."""
4     h = np.zeros(W_hh.shape[0]) if h0 is None else h0
5     out = []
6     for x_t in X:
7         h = np.tanh(W_xh @ x_t + W_hh @ h + b)
8         out.append(h)
9     return np.stack(out)
```

Mã nguồn 4. Lan truyền xuôi của Simple RNN theo công thức (1.2)

Hàm trên khớp `nn.RNN` của PyTorch (kiểm tra `rnn_forward`: đúng). PyTorch lưu hai vector bias b_{ih} và b_{hh} nên tầng `nn.RNN(8, 64)` có 4.736 tham số, nhiều hơn công thức ở Bảng 1.1 đúng $H = 64$. Khi so khớp, notebook cộng hai bias lại thành b .

1.5 Lan truyền ngược theo thời gian và hai vấn đề của gradient

Huấn luyện RNN dùng lan truyền ngược theo thời gian (BPTT) [2]: mở mạng ra T bước rồi lan truyền ngược như một mạng thường, cộng gradient của W_{hh} từ mọi bước. Gọi L là hàm mất mát tính từ h_T . Theo quy tắc chuỗi,

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^T W_{hh}^\top \text{diag}(1 - h_k^2). \quad (1.3)$$

Gradient đi từ h_T về h_t bị nhân với $T - t$ ma trận Jacobi. Nếu chuẩn của các ma trận này nhỏ hơn 1, tích co lại theo hàm mũ và gradient *triệt tiêu*; nếu lớn hơn 1, tích *bùng nổ* [3, 4]. Bỏ qua đạo hàm của $\tanh$ (luôn không vượt 1) để thấy riêng vai trò của W_{hh} : với vector v bất kỳ, $\|(W_{hh}^\top)^t v\|$ tăng hay giảm theo bán kính phổ ρ của W_{hh} .

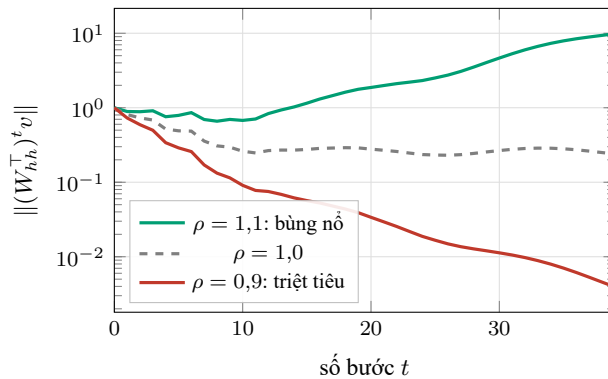
```

1 # a06/theory.py
2 def jacobian_product_norm(rho, T, n=32, seed=0):
3     """ $\|(W_{hh})^t v\| / \|v\|$  với  $W_{hh}$  ngẫu nhiên có bán kính phổ  $\rho$  (bỏ qua tanh để thấy riêng vai trò của  $W$ ).
4
5      $\rho < 1$ : suy giảm theo hàm mũ (triệt tiêu);  $\rho > 1$ : tăng theo hàm mũ (bùng nổ).
6     """
7     rng = np.random.default_rng(seed)
8     W = rng.standard_normal((n, n))
9     W *= rho / np.max(np.abs(np.linalg.eigvals(W)))
10    v = rng.standard_normal(n)
11    v /= np.linalg.norm(v)
12    norms = []
13    for _ in range(T):
14        norms.append(np.linalg.norm(v))
15        v = W.T @ v
16    return np.array(norms)

```

Mã nguồn 5. Chuẩn của tích Jacobi với bán kính phổ ρ cho trước

Hình 1.4 chạy hàm này với ba giá trị ρ . Sau 40 bước, $\rho = 0,9$ còn lại 0,0039, còn $\rho = 1,1$ đã lên 9,86. Chênh lệch chỉ 0,1 ở bán kính phổ thành chênh lệch hơn ba bậc độ lớn ở gradient.



Hình 1.4. Tích Jacobi co hoặc giãn theo hàm mũ tùy bán kính phổ của W_{hh}

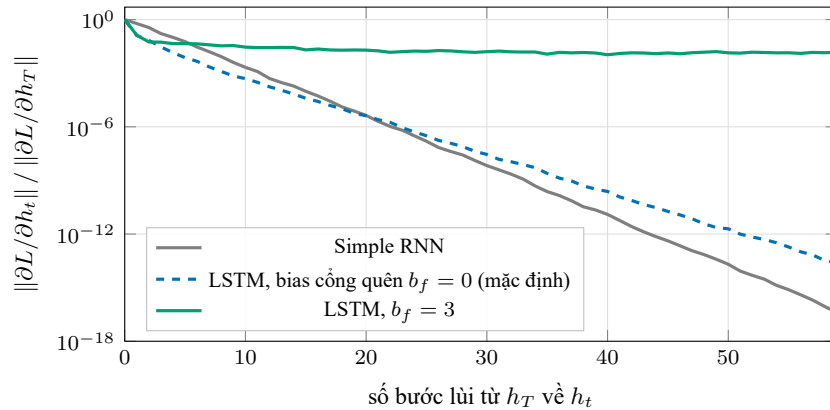
Thực tế có thêm tanh và trọng số khởi tạo ngẫu nhiên, nên notebook đo trực tiếp: dựng chuỗi ngẫu nhiên dài 60 bước, lấy $L = \sum h_T$, rồi dùng autograd đọc $\|\partial L / \partial h_t\|$ ở từng bước. Để giữ được gradient của từng h_t , hàm mở vòng lặp bằng RNNCell/LSTMCell thay cho lớp RNN gộp.

```

1 # a06/theory.py
2 def grad_norms_through_time(cell="rnn", T=100, D=8, H=32, seed=0, forget_bias=0.0):
3     """ $\|\partial L / \partial h_t\|$  với L chỉ phụ thuộc  $h_T$ , lấy bằng autograd trên một chuỗi ngẫu nhiên.
4
5     Mô vòng lặp bằng RNNCell/LSTMCell để giữ gradient của từng  $h_t$ . forget_bias > 0 đẩy cổng quên
6      $f_t$  về gần 1 (Jozefowicz và cộng sự, 2015), để đường cộng của cell state thực sự giữ gradient.
7     """
8     torch.manual_seed(seed)
9     layer = nn.RNNCell(D, H) if cell == "rnn" else nn.LSTMCell(D, H)
10    if cell == "lstm":
11        with torch.no_grad():
12            layer.bias_ih[H:2 * H] = forget_bias # khối thứ hai (i, f, g, o) là cổng quên
13            layer.bias_hh[H:2 * H] = 0.0
14    x = torch.randn(T, 1, D)
15    h = torch.zeros(1, H)
16    c = torch.zeros(1, H)
17    hs = []
18    for t in range(T):
19        if cell == "rnn":
20            h = layer(x[t], h)
21        else:
22            h, c = layer(x[t], (h, c))
23            h.retain_grad()
24            hs.append(h)
25    hs[-1].sum().backward()
26    return np.array([hh.grad.norm().item() for hh in hs])

```

Mã nguồn 6. Đo chuẩn gradient theo từng bước bằng autograd



Hình 1.5. Gradient của mất mát theo h_t , chuẩn hoá theo bước cuối (trục tung thang log)

Hình 1.5 cho một kết quả đáng chú ý. Với Simple RNN, lùi 20 bước thì gradient chỉ còn $4,33 \times 10^{-6}$ lần giá trị ở bước cuối; lùi 59 bước còn $4,06 \times 10^{-17}$. LSTM với khởi tạo mặc định của PyTorch cũng suy giảm gần như vậy: $4,16 \times 10^{-6}$ ở trễ 20. Lý do là bias cổng quên bằng 0 nên $f_t \approx 0,5$, và cell state bị nhân với khoảng 0,5 ở mỗi bước. Khi đặt bias cổng quên bằng 3 như Jozefowicz và cộng sự đề xuất [5], f_t tiến về gần 1 và gradient ở trễ 59 còn 0,0151, lớn hơn RNN hàng chục bậc. Vậy LSTM không tự động nhớ xa. Nó có một con đường để gradient đi qua, và mô hình phải học mở con đường đó.

1.6 Gradient clipping

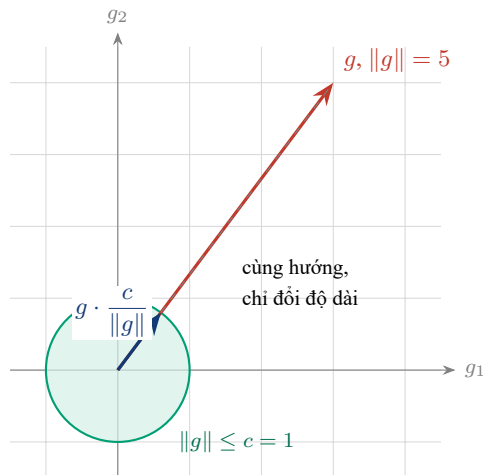
Triệt tiêu làm mô hình không học được phụ thuộc xa; bùng nổ thì nguy hiểm hơn, vì một bước cập nhật quá lớn có thể phá hỏng trọng số đã học. Cách chữa thông dụng là cắt theo chuẩn [4]: gộp mọi gradient thành một vector g , nếu $\|g\| > c$ thì thay bằng $g \cdot c / \|g\|$. Độ dài bị chặn, còn hướng giữ nguyên (Hình 1.6). Cả 16 mô hình của bài dùng $c = 1$.

```

1 # a06/theory.py
2 def clip_grad_norm(grads, max_norm):
3     """Co toàn bộ gradient về chuẩn max_norm nếu vượt, GIỮ NGUYÊN HƯỚNG. Trả về (grads mới, chuẩn cũ)."""
4     total = np.sqrt(sum(np.sum(g ** 2) for g in grads))
5     scale = min(1.0, max_norm / (total + 1e-6))
6     return [g * scale for g in grads], total

```

Mã nguồn 7. Cắt gradient theo chuẩn, giữ hướng



Hình 1.6. Gradient clipping chỉ đổi độ dài của g , không đổi hướng

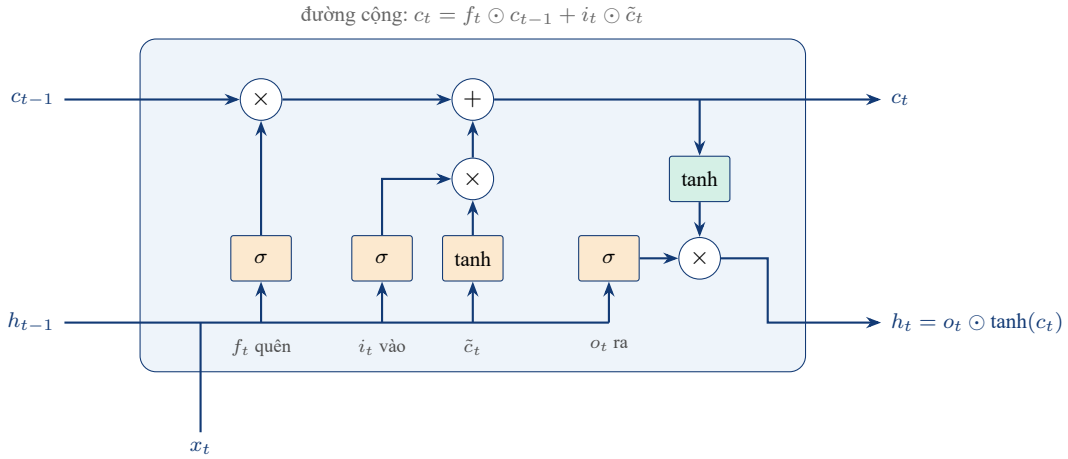
Trên hai ma trận gradient ngẫu nhiên có tổng chuẩn 72,18, hàm trả về chuẩn 1 và khớp `nn.utils.clip_grad_norm_` (đúng).

1.7 LSTM

LSTM [6] thêm một trạng thái thứ hai là cell state c_t và ba cổng điều khiển nó. Với $z_t = [h_{t-1}; x_t]$,

$$\begin{aligned}
 f_t &= \sigma(W_f z_t + b_f), & i_t &= \sigma(W_i z_t + b_i), & o_t &= \sigma(W_o z_t + b_o), \\
 \tilde{c}_t &= \tanh(W_c z_t + b_c), & c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t).
 \end{aligned} \tag{1.4}$$

Cổng quên f_t quyết định giữ bao nhiêu của c_{t-1} , cổng vào i_t quyết định ghi bao nhiêu thông tin mới $\tilde{c}_t$, cổng ra o_t quyết định để lộ bao nhiêu ra h_t . Điểm mấu chốt là phép cộng ở dòng c_t (Hình 1.7): $\partial c_t / \partial c_{t-1} = f_t$, không có ma trận trọng số và không có tanh trên đường đi. Khi f_t gần 1, gradient đi qua nhiều bước gần như nguyên vẹn.



Hình 1.7. Tế bào LSTM: đường cộng của cell state ở trên, bốn phép biến đổi có cổng ở dưới

```

1 # a06/theory.py
2 def lstm_cell(x, h, c, W_ih, W_hh, b_ih, b_hh):
3     """Một bước LSTM. W_ih (4H, D), W_hh (4H, H) xếp khối theo thứ tự cổng i, f, g, o như PyTorch."""
4     z = W_ih @ x + b_ih + W_hh @ h + b_hh
5     i, f, g, o = np.split(z, 4)
6     i, f, g, o = sigmoid(i), sigmoid(f), np.tanh(g), sigmoid(o)
7     c_new = f * c + i * g          # cell state: cộng có cổng, gradient đi thẳng qua f
8     h_new = o * np.tanh(c_new)
9     return h_new, c_new
    
```

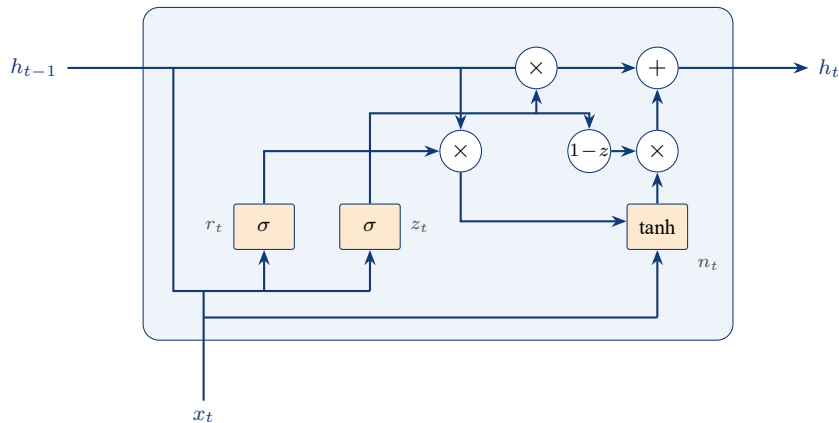
Mã nguồn 8. Một bước LSTM theo (1.4), thứ tự khối cổng như PyTorch

1.8 GRU

GRU [7] gộp cell state vào h_t và chỉ dùng hai cổng: cổng reset r_t quyết định dùng bao nhiêu trạng thái cũ khi tạo ứng viên n_t , cổng cập nhật z_t hoà trộn trạng thái cũ với ứng viên,

$$\begin{aligned}
 r_t &= \sigma(W_r z_t + b_r), & z_t &= \sigma(W_z z_t + b_z), \\
 n_t &= \tanh(W_{in} x_t + r_t \odot (W_{hn} h_{t-1})), & h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1}.
 \end{aligned}
 \quad (1.5)$$

Dòng cuối là một tổ hợp lười: khi $z_t \rightarrow 1$ thì $h_t \approx h_{t-1}$, gradient đi thẳng qua giống đường cộng của LSTM (Hình 1.8).



Hình 1.8. Tế bào GRU với hai cổng

```

1 # a06/theory.py
2 def gru_cell(x, h, W_ih, W_hh, b_ih, b_hh):
3     """Một bước GRU, thứ tự khối r, z, n như PyTorch; h' là tổ hợp lỗi của h cũ và ứng viên n."""
4     gi = W_ih @ x + b_ih
5     gh = W_hh @ h + b_hh
6     ir, iz, in_ = np.split(gi, 3)
7     hr, hz, hn = np.split(gh, 3)
8     r = sigmoid(ir + hr)
9     z = sigmoid(iz + hz)
10    n = np.tanh(in_ + r * hn)
11    return (1 - z) * n + z * h

```

Mã nguồn 9. Một bước GRU theo (1.5)

Bảng 1.3 so hai tế bào với cùng $D = 8$, $H = 64$. Số tham số đọc từ PyTorch.

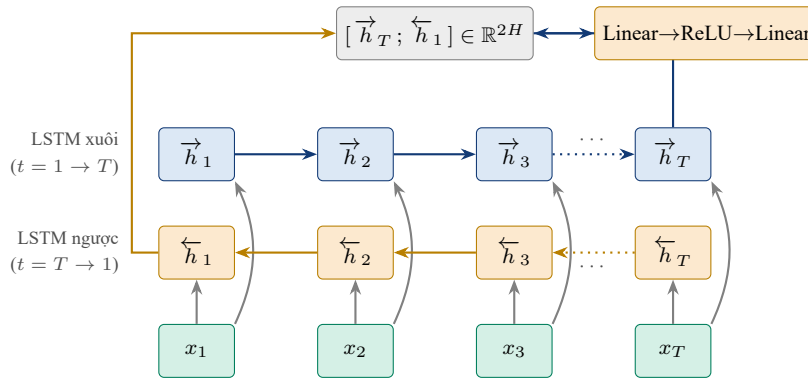
Bảng 1.3. So sánh Simple RNN, LSTM, GRU và BiLSTM ($D = 8$, $H = 64$, tham số của riêng tầng hồi quy trong PyTorch)

Tế bào	Số khối trọng số	Trạng thái	Cổng	Tham số
Simple RNN	1	h_t	không	4.736
LSTM	4	h_t, c_t	f, i, o	18.944
GRU	3	h_t	r, z	14.208
BiLSTM	2×4	$\vec{h}_t, \overleftarrow{h}_t$	f, i, o mỗi chiều	37.888

LSTM hay GRU? GRU có ba khối trọng số thay vì bốn, nên ít hơn đúng một phần tư tham số: tỉ lệ đo được là 0,75. Về lý thuyết GRU cũng ít phép tính hơn mỗi bước. Trên thực tế tốc độ còn phụ thuộc cách thư viện cài đặt: ở phép đo của Chương 4 trên CPU một luồng, GRU của PyTorch mất 1,48 ms mỗi mẫu, trong khi LSTM chỉ mất 0,3945 ms. Về chất lượng, hai tế bào gần như ngang nhau trên cả hai tập (Chương 3), đúng với quan sát chung của Jozefowicz và cộng sự [5]. LSTM có cell state riêng nên tách được “nhớ” với “xuất ra”; GRU gộp hai việc đó vào h_t .

1.9 LSTM hai chiều

BiLSTM [8] chạy hai LSTM độc lập, một đọc từ $t = 1$ đến T , một đọc từ T về 1, rồi nối hai trạng thái cuối: $[\vec{h}_T; \overleftarrow{h}_1] \in \mathbb{R}^{2H}$ (Hình 1.9). Notebook kiểm tra cách nối này trên nn.LSTM(bidirectional=True): đầu ra có shape (4, 30, 128), nửa đầu ở bước cuối cùng $\vec{h}_T$ và nửa sau ở bước đầu cùng $\overleftarrow{h}_1$ (đúng).



Hình 1.9. BiLSTM: vector tóm tắt là trạng thái cuối của hai chiều đọc

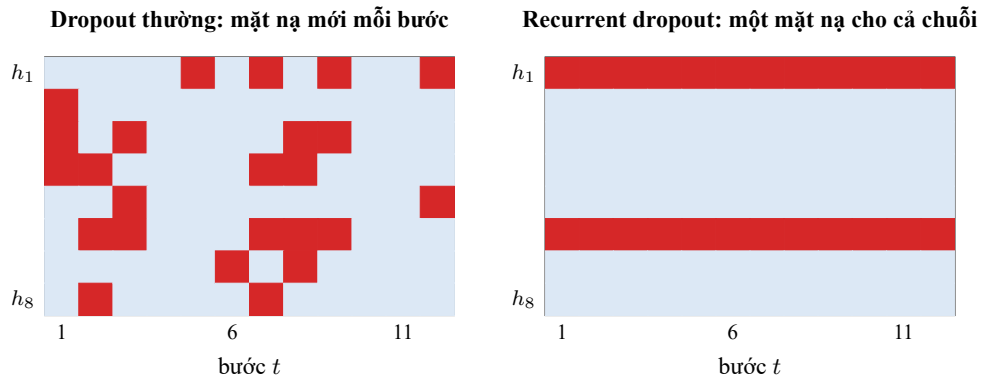
Với bài dự báo giá, chiều ngược không làm lộ tương lai. Nó chỉ đọc lại 30 phiên của cửa sổ theo thứ tự ngược,

mọi phiên đều trước ngày cần dự báo. Chiều ngược thêm một góc nhìn khác lên cùng quá khứ, không thêm thông tin mới.

1.10 Điều chuẩn cho mạng hồi quy

Dropout thường đổi mặt nạ ở mỗi bước. Áp lên trạng thái h_t , nó xoá một phần trí nhớ khác nhau ở mỗi bước, và thông tin khó đi xa. Gal và Ghahramani [9] đề xuất dùng *một* mặt nạ cho cả chuỗi (recurrent dropout): đơn vị bị tắt thì tắt suốt chuỗi, đơn vị còn lại mang thông tin liền mạch (Hình 1.10, $p = 30,00\%$):

$$m \sim \text{Bernoulli}(1 - p)^H \text{ (lấy một lần cho cả chuỗi)}, \quad h_t = \tanh(W_{xh}x_t + W_{hh}(m \odot h_{t-1}) + b). \quad (1.6)$$



Hình 1.10. Mặt nạ dropout thường và recurrent dropout; ô đỏ là đơn vị bị tắt

```
1 # a06/theory.py
2 def recurrent_dropout_masks(T, H, p, seed=0):
3     """Dropout thường: mặt nạ mới ở mỗi bước. Recurrent dropout (Gal & Ghahramani): một mặt nạ cho cả chuỗi."""
4     rng = np.random.default_rng(seed)
5     standard = (rng.random((T, H)) >= p).astype(float)
6     fixed = np.tile((rng.random(H) >= p).astype(float), (T, 1))
7     return standard, fixed
```

Mã nguồn 10. Hai kiểu mặt nạ dropout cho chuỗi

BatchNorm của các bài trước lấy thống kê theo lô và theo từng bước, không hợp với chuỗi có độ dài thay đổi. LayerNorm [10] chuẩn hoá theo chiều đặc trưng của *từng mẫu*, nên dùng được ở mọi bước và mọi kích thước lô:

$$\mu_t = \frac{1}{H} \sum_{i=1}^H a_{t,i}, \quad \sigma_t^2 = \frac{1}{H} \sum_{i=1}^H (a_{t,i} - \mu_t)^2, \quad \text{LN}(a_t) = \gamma \odot \frac{a_t - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}} + \beta. \quad (1.7)$$

Hàm dưới khớp nn.LayerNorm (đúng).

```
1 # a06/theory.py
2 def layer_norm(x, gamma, beta, eps=1e-5):
3     """Chuẩn hoá theo chiều đặc trưng của TỪNG MẪU (không phụ thuộc Lô, hợp với RNN)."""
4     mu = x.mean(axis=-1, keepdims=True)
5     var = x.var(axis=-1, keepdims=True)
6     return gamma * (x - mu) / np.sqrt(var + eps) + beta
```

Mã nguồn 11. Layer Normalization theo từng mẫu

Các mô hình ở Chương 3 chỉ có một tầng hồi quy 64 đơn vị và dừng sớm theo validation, nên bài không dùng dropout hay LayerNorm. Hai kỹ thuật được trình bày ở đây vì chúng là công cụ chuẩn khi mô hình sâu hơn.

Chương 2

Hai tập dữ liệu chuỗi thực tế

Bài dùng hai tập dữ liệu thật, đủ lớn và thuộc hai loại bài toán khác nhau. AMZN là chuỗi giá cổ phiếu, một bài hỏi quy: dự báo giá đóng cửa của phiên kế tiếp. KKBox Churn là log nghe nhạc theo ngày của người dùng một dịch vụ trả phí, một bài phân loại: người dùng này có bỏ dịch vụ hay không. Cả hai đều đưa về tensor $B \times T \times 8$ như Chương 1, nên cùng một lớp mô hình chạy được trên cả hai.

2.1 Giá cổ phiếu AMZN

2.1.1 Nguồn và quy mô

Dữ liệu ngày của Amazon.com, Inc. (NASDAQ: AMZN) được tải từ Yahoo Finance [11] bằng thư viện yfinance, gồm 6.885 phiên từ 1997-05-15 (phiên đầu sau IPO) đến 2024-09-24. Mỗi phiên có giá mở cửa, cao nhất, thấp nhất, đóng cửa và khối lượng. Giá đã điều chỉnh theo các lần chia tách cổ phiếu, gần nhất là lần chia 20:1 năm 2022. Tập CSV được lưu trong kho mã nguồn để lần chạy sau dùng đúng dữ liệu này, không phụ thuộc việc Yahoo cập nhật lại.

```
1 # a06/data_stock.py
2 def add_features(df):
3     """Đặc trưng tài chính; các dòng đầu thiếu MA/RSI bị bỏ."""
4     df = df.copy()
5     df["Return"] = np.log(df.Close / df.Close.shift(1))
6     df["MA20"] = df.Close.rolling(20).mean()
7     df["MA50"] = df.Close.rolling(50).mean()
8     df["Volatility20"] = df.Return.rolling(20).std() * np.sqrt(252)
9     df["RSI14"] = rsi(df.Close)
10    return df.dropna()
```

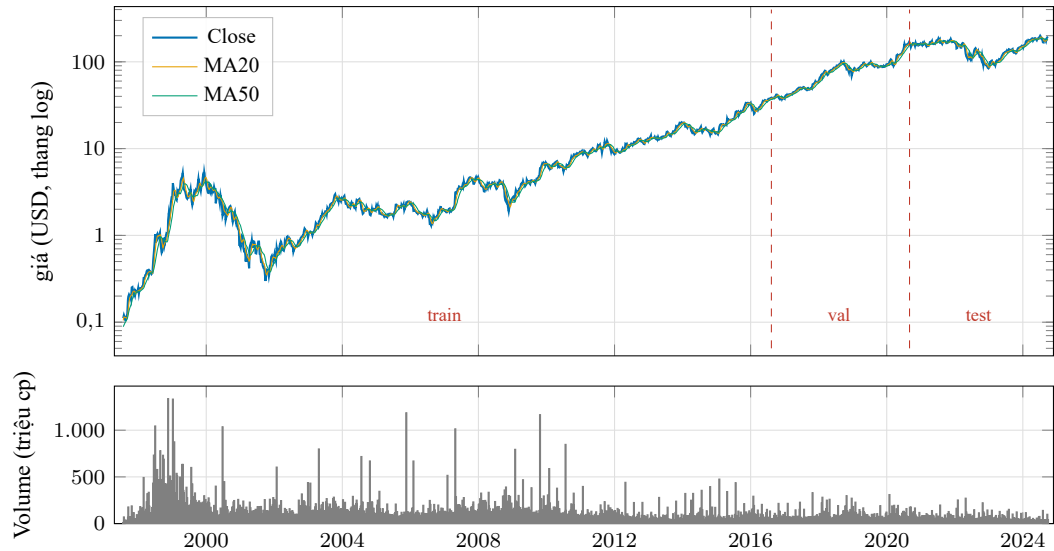
Mã nguồn 12. Thêm đặc trưng tài chính từ giá và khối lượng

Tám đặc trưng đưa vào mô hình là Close, Open, High, Low, Volume, Return (log-return $r_t = \ln(P_t/P_{t-1})$), MA20 (trung bình 20 phiên) và RSI14 (chỉ số sức mạnh tương đối của Wilder [12]). MA50 và Volatility20 chỉ dùng để phân tích. Bỏ 6.885 – 6.836 phiên đầu chưa đủ 50 phiên để tính MA50, còn lại 6.836 phiên. Bảng 2.1 cho thống kê mô tả.

Bảng 2.1. Thống kê mô tả 10 đặc trưng của AMZN, 6.836 phiên

Đặc trưng	TB	ĐLC	Min	Q25	Trung vị	Q75	Max	Kurtosis
Close (USD)	38,42	54,70	0,10	2,12	8,91	58,30	200,00	0,60
Open (USD)	38,43	54,73	0,10	2,11	8,92	58,44	200,09	0,60
High (USD)	38,88	55,35	0,10	2,16	9,01	58,79	201,20	0,59
Low (USD)	37,93	54,04	0,10	2,08	8,81	58,00	199,05	0,61
Volume (triệu cp)	137,59	137,38	9,74	64,79	101,70	156,28	2.086,58	34,24
Return (%)	0,11	3,46	−28,46	−1,30	0,05	1,46	29,62	9,71
MA20 (USD)	38,16	54,39	0,10	2,12	8,96	57,45	193,60	0,59
MA50 (USD)	37,77	53,96	0,09	2,12	8,83	53,46	187,88	0,60
Volatility20 (năm)	0,46	0,30	0,08	0,24	0,37	0,57	1,59	1,52
RSI14	53,51	12,06	17,75	44,77	53,81	62,02	94,21	−0,41

TB: trung bình; ĐLC: độ lệch chuẩn; kurtosis là độ nhọn dư (phân phối chuẩn bằng 0).

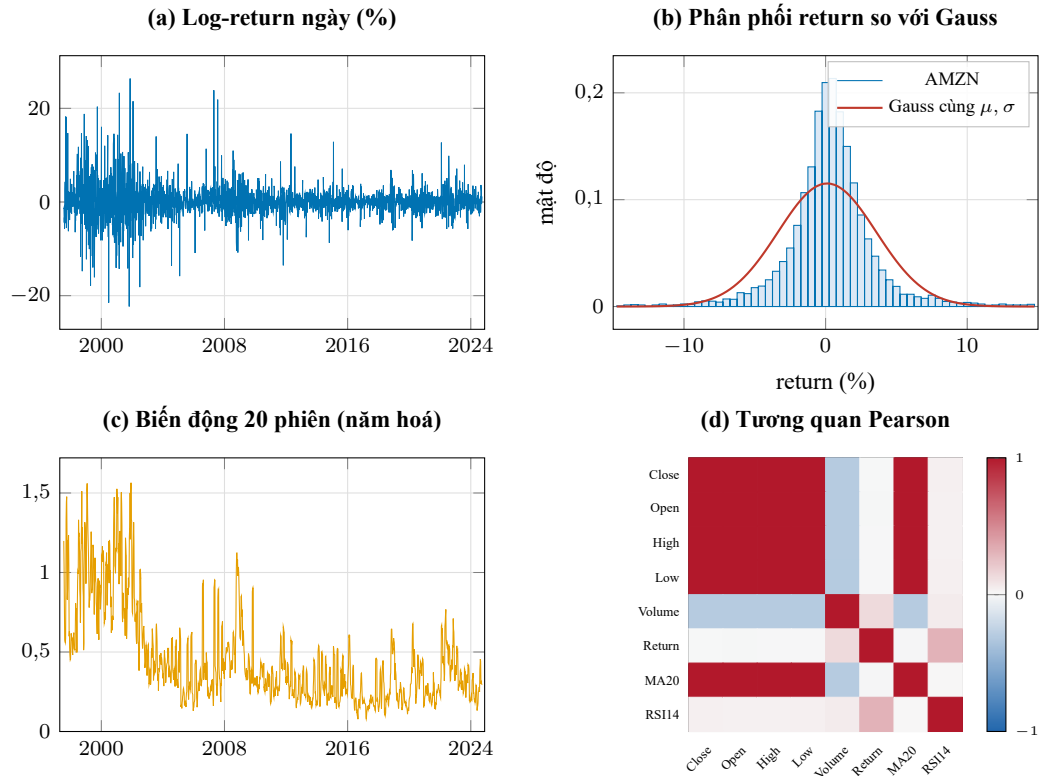


Hình 2.1. Giá đóng cửa AMZN (thang log) với MA20, MA50 và khối lượng; vạch đứt là ranh giới train, val, test

Hình 2.1 vẽ giá trên thang log. Trong 27 năm giá tăng hơn một nghìn lần, với hai lần sụt mạnh: bong bóng dot-com 2000–2001 và đợt điều chỉnh 2022. Thang tuyến tính sẽ ép toàn bộ giai đoạn trước 2010 thành một đường nằm sát trục hoành.

2.1.2 Return, phân phối và biến động

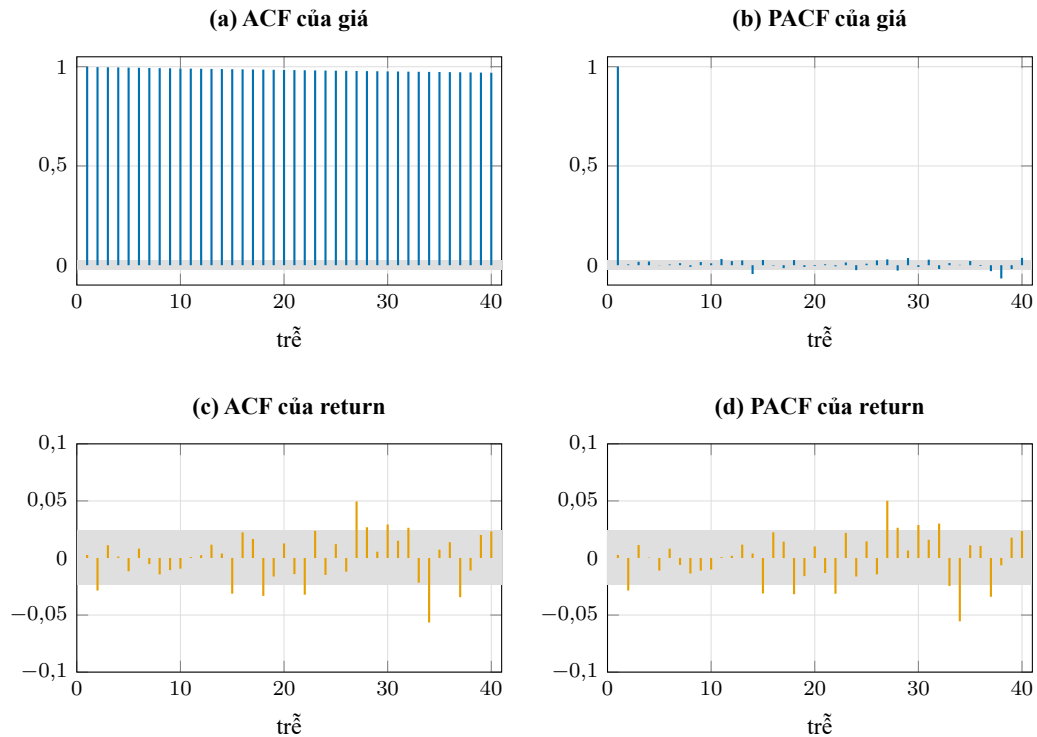
Hình 2.2 xem xét log-return. Return trung bình 0,0011 mỗi phiên, độ lệch chuẩn 0,0346. Phiên giảm mạnh nhất là 2001-07-24 ($r = -0,2846$), tăng mạnh nhất là 2001-11-26 ($r = 0,2962$), đều trong giai đoạn dot-com. Phân phối có đuôi dày: độ nhọn dư (excess kurtosis) là 9,7, trong khi phân phối chuẩn có độ nhọn dư bằng 0. Tỷ lệ phiên lệch quá 4 độ lệch chuẩn là 0,99%, nếu phân phối chuẩn thì chỉ 0,01%. Biến động (ô c) tụ thành cụm: những giai đoạn dao động mạnh kéo dài nhiều tháng (hiệu ứng ARCH). Biến động 20 phiên năm hoá đạt đỉnh 1,59 vào 2001-12-05 khi bong bóng dot-com vỡ, 1,18 vào 2008-10-15 trong khủng hoảng tài chính 2008, và 0,6796 vào 2020-03-25 khi COVID-19 bùng phát, trong khi trung vị của giai đoạn yên ả 2012–2019 chỉ là 0,2316. Khối lượng giao dịch tương quan dương với biến động ($r = 0,4882$), còn mức giá tương quan âm với biến động ($r = -0,3441$): giai đoạn giá thấp đầu những năm 2000 cũng là giai đoạn dao động mạnh nhất. Ô d cho thấy năm cột giá gần như trùng nhau (tương quan gần 1). Chúng mang thông tin về mức giá, còn Return và RSI14 mang thông tin về thay đổi.



Hình 2.2. Log-return, phân phối return so với phân phối chuẩn, biến động 20 phiên và tương quan giữa 8 đặc trưng

2.1.3 Tính dừng: kiểm định ADF và tự tương quan

Kiểm định Augmented Dickey–Fuller [13] có giả thuyết không là chuỗi có nghiệm đơn vị (không dừng). Với giá đóng cửa, thống kê ADF là 1,25, $p = 0,9963$, không bác bỏ được giả thuyết: giá không dừng. Với return, thống kê là $-13,99$, $p = 4,08 \times 10^{-26}$: return dừng. Hình 2.3 cho cùng kết luận. ACF của giá giảm rất chậm, ở trễ 40 vẫn còn 0,9696. ACF của return gần như nằm trong dải tin cậy $\pm 0,0237$; chỉ 11 trong 40 trễ vượt ra ngoài và đều rất nhỏ.



Hình 2.3. ACF và PACF 40 trễ của giá và của return; dải xám là khoảng tin cậy 95 %

Hai kết quả này quyết định cách đặt bài toán. Giá không dừng, nên mô hình học trên giá 1997–2016 sẽ gặp mức giá hoàn toàn mới ở tập test: giá lớn nhất của test gấp 5,19 lần giá lớn nhất của train. Một mô hình dự báo thẳng mức giá đã chuẩn hoá Min-Max theo train sẽ phải ngoại suy ra ngoài khoảng $[0, 1]$ mà nó từng thấy. Return thì dừng, nhưng gần như không có tự tương quan, tức là rất khó dự báo từ quá khứ của chính nó. Đó là điều giả thuyết thị trường hiệu quả [14] dự đoán, và Chương 4 sẽ thấy các mô hình va vào đúng giới hạn này.

2.1.4 Cửa sổ trượt và chống rò rỉ

Bài dự báo log-return phiên kế tiếp r_{t+1} từ cửa sổ 30 phiên, rồi quy đổi ra giá $\hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}}$ để báo sai số bằng USD. Trong mỗi cửa sổ, năm cột giá chia cho Close của ngày cuối cửa sổ, nên mô hình chỉ thấy giá tương đối (ví dụ “MA20 cao hơn giá hiện tại 3 %”), không thấy mức giá tuyệt đối. Volume lấy log.

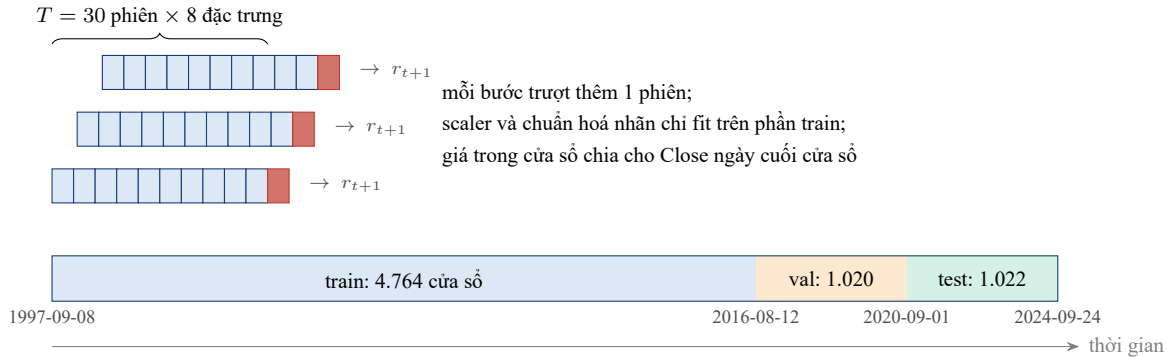
```

1 # a06/data_stock.py
2 def make_windows(df, window=WINDOW):
3     """Cửa sổ trượt:  $X[i] = 30$  phiên kết thúc ở  $t$ , nhãn = log-return của phiên  $t+1$ .
4
5     Giá trong cửa sổ chia cho Close ngày  $t$  (bất biến theo mức giá), Volume lấy log.
6     Trả về  $X$  thô (chưa z-score), nhãn, giá  $P_t$ , giá  $P_{t+1}$  và ngày của  $P_{t+1}$ .
7     """
8     vals = _feature_values(df)
9     close = df.Close.to_numpy(np.float64)
10    X, y, p_now, p_next, dates = [], [], [], [], []
11    for t in range(window - 1, len(df) - 1):
12        X.append(_relative_prices(vals[t - window + 1:t + 1], close[t]))
13        y.append(np.log(close[t + 1] / close[t]))
14        p_now.append(close[t])
15        p_next.append(close[t + 1])
16        dates.append(df.index[t + 1])
17    return np.stack(X), np.array(y), np.array(p_now), np.array(p_next), pd.DatetimeIndex(dates)

```

Mã nguồn 13. Cửa sổ trượt với nhãn là return phiên kế tiếp

Tổng cộng 6.806 cửa sổ, chia theo thời gian 70/15/15 và không xáo trộn (Hình 2.4). Tập test là bốn năm cuối, từ 2020-09-01 đến 2024-09-24. Mọi thống kê chuẩn hoá (trung bình, độ lệch chuẩn của từng đặc trưng và của nhãn) chỉ tính trên phần train. Notebook 01 kiểm tra bằng assert rằng ngày cuối của train đứng trước ngày đầu của val, và ngày cuối của val đứng trước ngày đầu của test.



Hình 2.4. Cửa sổ trượt và cách chia theo thời gian của AMZN

```

1 # a06/data_stock.py
2 class WindowScaler:
3     """z-score từng đặc trưng cho X và z-score cho nhãn, cả hai chỉ fit trên cửa sổ train."""
4
5     def fit(self, X, y):
6         flat = X.reshape(-1, X.shape[-1])
7         self.mean_, self.std_ = flat.mean(axis=0), flat.std(axis=0) + 1e-8
8         self.y_mean_, self.y_std_ = float(y.mean()), float(y.std())
9         return self
10
11     def transform(self, X):
12         return ((X - self.mean_) / self.std_).astype(np.float32)
13
14     def transform_y(self, y):
15         return ((y - self.y_mean_) / self.y_std_).astype(np.float32)
16
17     def inverse_y(self, z):
18         return np.asarray(z, dtype=np.float64) * self.y_std_ + self.y_mean_
19
20     def state(self):
21         return {"mean": self.mean_.tolist(), "std": self.std_.tolist(),
22               "y_mean": self.y_mean_, "y_std": self.y_std_}

```

Mã nguồn 14. Chuẩn hoá đặc trưng và nhãn, fit chỉ trên train

Độ lệch chuẩn của return ở ba phần cũng khác nhau: 0,0391 ở train (có cả giai đoạn dot-com), 0,0184 ở val và 0,0225 ở test. Mô hình học trên một thị trường biến động mạnh hơn nhiều so với thị trường nó được đánh giá.

2.2 KKBBox Churn

2.2.1 Nguồn và định nghĩa nhãn

KKBBox là dịch vụ nghe nhạc trực tuyến lớn ở Đài Loan. Năm 2017, KKBBox và hội nghị WSDM 2018 mở một cuộc thi trên Kaggle [15] với dữ liệu thật của người dùng đã ẩn danh. Bài dùng hai tập:

- train_v2.csv: 970.960 người dùng, nhãn is_churn bằng 1 nếu người dùng *không* gia hạn gói thuê bao trong 30 ngày sau khi gói hết hạn vào tháng 3/2017;

- `user_logs_v2.csv` (1,4 GB, 18.396.362 dòng): mỗi dòng là một ngày nghe của một người dùng trong tháng 3/2017, gồm số bài nghe dưới 25 %, 25–50 %, 50–75 %, 75–98,5 % và trên 98,5 % độ dài bài (`num_25` đến `num_100`), số bài khác nhau (`num_unq`) và tổng số giây nghe (`total_secs`).

2.2.2 Lấy mẫu và dựng chuỗi

Chỉ 754.551 người có cả nhãn lẫn ít nhất một ngày log. Từ đó bài lấy mẫu phân tầng 100.000 người (`seed 42`). Tập log không được đọc trọn vào RAM: `pyarrow` đọc theo khối 64 MB, lượt thứ nhất lấy tập người có log, lượt thứ hai giữ lại 1.795.032 dòng của người trong mẫu. Cả quá trình mất khoảng một phút.

```

1 # a06/data_churn.py
2 def sample_users(n=N_USERS, seed=SEED):
3     """Người có nhãn VÀ có ít nhất một dòng log; lấy mẫu phân tầng theo is_churn, giữ nguyên tỉ lệ nhãn."""
4     labels = pacsv.read_csv(KKBOX / "train_v2.csv").to_pandas()
5     seen = set()
6     for batch in _stream_logs():
7         seen.update(pc.unique(batch.column("msno")).to_pylist())
8     has_log = labels[labels.msno.isin(seen)].reset_index(drop=True)
9     rng = np.random.default_rng(seed)
10    frac = n / len(has_log)
11    idx = []
12    for _, g in has_log.groupby("is_churn"):
13        k = int(round(len(g) * frac))
14        idx.extend(rng.choice(g.index.to_numpy(), size=k, replace=False))
15    sample = has_log.loc[np.sort(idx)].reset_index(drop=True)
16    info = {"n_labeled": len(labels), "n_with_log": len(has_log), "n_sample": len(sample),
17           "churn_rate_labeled": float(labels.is_churn.mean()),
18           "churn_rate_with_log": float(has_log.is_churn.mean()),
19           "churn_rate_sample": float(sample.is_churn.mean())}
20    return sample, info

```

Mã nguồn 15. Lấy mẫu phân tầng những người vừa có nhãn vừa có log

Mỗi người dùng thành một ma trận 31×8 : bảy cột log lấy $\log(1 + x)$ vì phân phối đuôi rất dài, cột thứ tám là cờ active (ngày đó có nghe hay không). Ngày không có log để 0. Một ngày có thể có nhiều dòng (nhiều thiết bị) nên được cộng dồn. 379 dòng có `total_secs` âm hoặc vượt 86.400 giây (dài hơn một ngày) là lỗi ghi log và được kẹp về $[0, 86.400]$.

```

1 # a06/data_churn.py
2 def build_sequences(sample):
3     """Gom Log của các người dùng trong mẫu thành tensor (N, 31, 8).
4
5     Một ngày có thể có nhiều dòng (nhiều thiết bị) nên cộng dồn theo (người, ngày). Đặc trưng đếm
6     và số giây lấy log1p vì phân phối đuôi rất dài; ngày không có Log để 0 và cờ active = 0.
7     """
8     pos = {m: i for i, m in enumerate(sample.msno)}
9     wanted = pa.array(list(pos))
10    X = np.zeros((len(sample), T_DAYS, len(FEATURES)), dtype=np.float64)
11    n_rows = n_kept = n_clipped = 0
12    for batch in _stream_logs():
13        n_rows += batch.num_rows
14        batch = batch.filter(pc.is_in(batch.column("msno"), value_set=wanted))
15        if batch.num_rows == 0:
16            continue
17        df = batch.to_pandas()
18        n_kept += len(df)
19        bad = (df.total_secs < 0) | (df.total_secs > MAX_SECS)
20        n_clipped += int(bad.sum())
21        df["total_secs"] = df.total_secs.clip(0, MAX_SECS)
22        u = df.msno.map(pos).to_numpy()
23        d = (df.date - FIRST_DAY).to_numpy()
24        np.add.at(X, (u, d, slice(0, len(LOG_COLS))), df[LOG_COLS].to_numpy(np.float64))
25    X[:, :, -1] = (X[:, :, :len(LOG_COLS)].sum(axis=2) > 0)
26    X[:, :, :len(LOG_COLS)] = np.log1p(X[:, :, :len(LOG_COLS)])
27    info = {"log_rows": n_rows, "log_rows_kept": n_kept, "secs_clipped": n_clipped}
28    return X.astype(np.float32), info

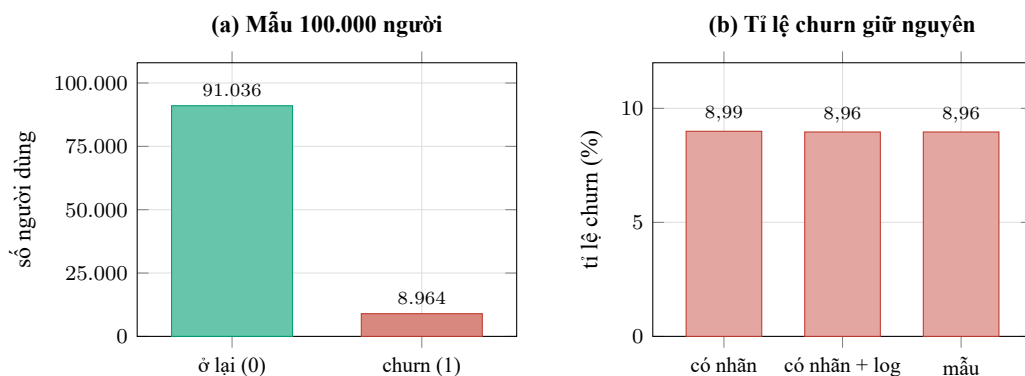
```

Mã nguồn 16. Gom log theo ngày thành tensor ($N, 31, 8$)

Chia 70/15/15 phân tầng theo nhãn: 70.000 người train, 15.000 val và 15.000 test. Chuẩn hoá z-score từng đặc trưng, thống kê tính trên mọi ngày của mọi người trong train.

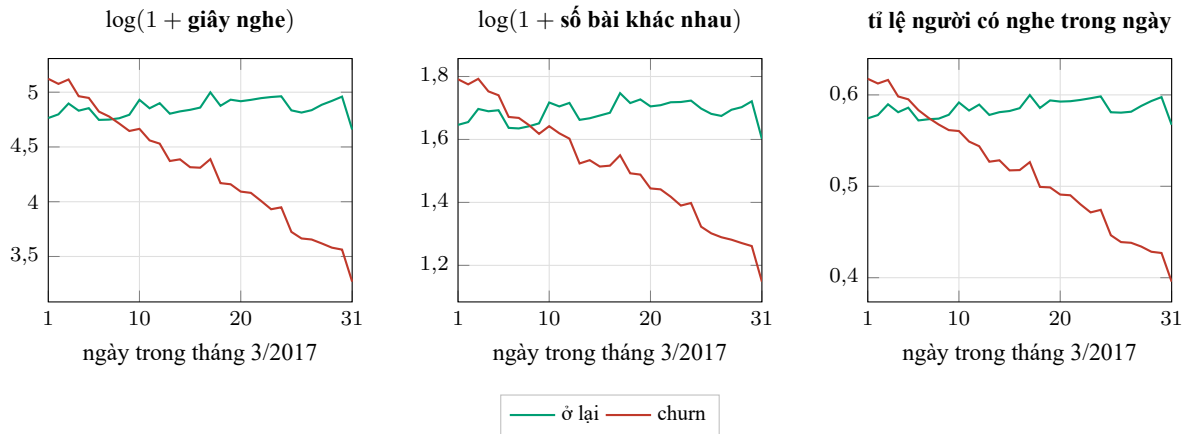
2.2.3 Phân tích dữ liệu

Nhãn lệch mạnh: 8.964 người churn trên 100.000, tức 8,96 % (Hình 2.5). Tỷ lệ này gần như giữ nguyên qua hai bước lọc (8,99 % trên toàn bộ người có nhãn), nên mẫu đại diện được tập gốc. Một mô hình luôn đoán “ở lại” đã đạt accuracy 100 – 8,96 %, vì vậy accuracy không phải thước đo phù hợp; bài dùng F1, Recall, ROC-AUC và PR-AUC.

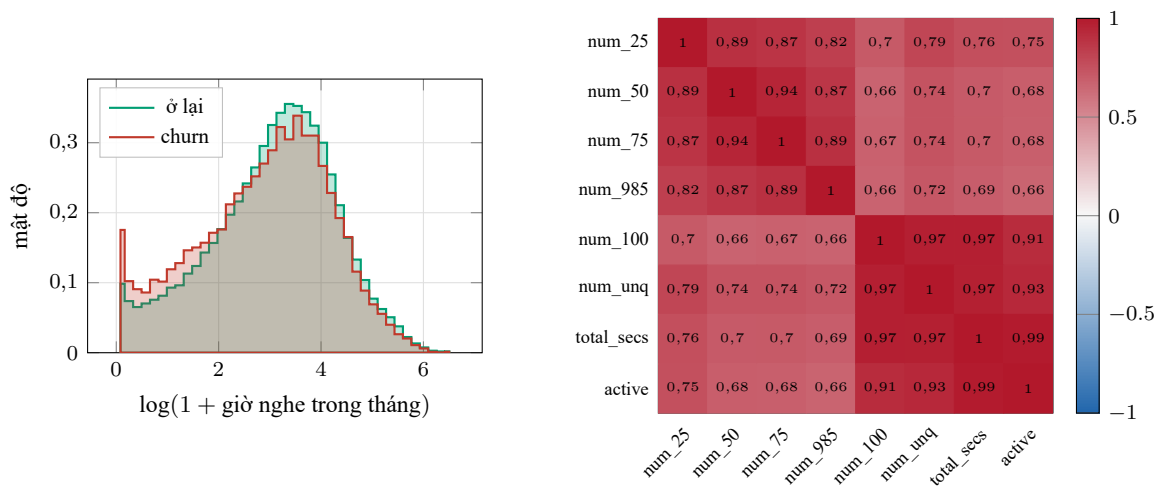


Hình 2.5. Phân bố nhãn trong mẫu và tỉ lệ churn qua các bước lọc

Hình 2.6 là kết quả thú vị nhất của phần phân tích. Đầu tháng 3, nhóm sẽ churn nghe *nhiều hơn* nhóm ở lại ở cả ba đặc trưng. Trong tuần thứ hai (giữa ngày 6 và ngày 11) hai đường cắt nhau, rồi nhóm churn giảm đều đến cuối tháng, còn nhóm ở lại gần như đi ngang. Tính trung bình cả tháng, người churn có 16,01 ngày nghe, người ở lại có 18,14 ngày. Tín hiệu nằm ở *xu hướng* theo thời gian chứ không nằm ở mức trung bình, và đó là loại tín hiệu mô hình hồi quy có thể đọc được mà mô hình trên đặc trưng tổng hợp sẽ bỏ lỡ.



Hình 2.6. Quỹ đạo trung bình 31 ngày của nhóm ở lại và nhóm churn



Hình 2.7. Trái: phân phối tổng giờ nghe trong tháng. Phải: tương quan giữa 8 đặc trưng

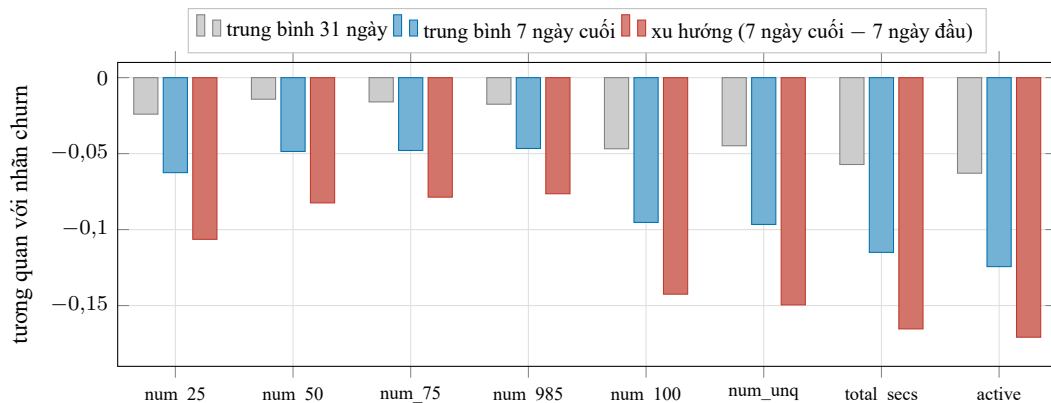
Hình 2.7 (trái) cho thấy phân phối tổng giờ nghe của hai nhóm chồng lên nhau gần hết; trung vị là 19,95 giờ với nhóm churn và 23,32 giờ với nhóm ở lại. Các đặc trưng tương quan mạnh với nhau (phải): num_100 và total_secs có hệ số 0,967. Bảng 2.2 so hai nhóm bằng kiểm định t của Welch [16]. Mọi khác biệt đều có ý nghĩa thống kê vì mẫu lớn, nhưng độ chênh của trung bình nhỏ so với độ phân tán trong từng nhóm. Hành vi nghe chỉ phân biệt được một phần người churn; phần còn lại phụ thuộc thông tin không có trong log, như phương thức thanh toán hay gói cước. Chương 3 sẽ đo mức trần này.

Bảng 2.2. Kiểm định t Welch giữa nhóm ở lại và nhóm churn, trung bình 31 ngày của từng người

Đặc trưng (TB 31 ngày)	Loyal	Churn	Chênh (%)	t Welch	p
num_25	0,742	0,688	-7,3	+7,54	$5,19 \times 10^{-14}$
num_50	0,332	0,316	-4,9	+4,42	$1,01 \times 10^{-5}$
num_75	0,258	0,244	-5,7	+5,05	$4,58 \times 10^{-7}$
num_985	0,275	0,257	-6,6	+5,59	$2,36 \times 10^{-8}$
num_100	1,643	1,456	-11,4	+14,87	$< 10^{-300}$
num_unq	1,688	1,513	-10,3	+14,00	$< 10^{-300}$
total_secs	4,859	4,296	-11,6	+17,75	$< 10^{-300}$
active	0,585	0,516	-11,7	+19,37	$< 10^{-300}$

Đặc trưng đếm và số giây đã lấy $\log(1+x)$; active là tỉ lệ ngày có nghe.

Để kiểm chứng nhận định “tín hiệu nằm ở xu hướng”, notebook 02 tính tương quan của từng đặc trưng với nhân churn theo ba cách tóm tắt chuỗi: trung bình cả tháng, trung bình 7 ngày cuối, và xu hướng (7 ngày cuối trừ 7 ngày đầu). Hình 2.8 cho thấy cột xu hướng mạnh hơn hẳn ở cả tám đặc trưng. Với active, độ lớn tương quan tăng từ -0,0629 (trung bình) lên -0,1244 (7 ngày cuối) và -0,1708 (xu hướng); với total_secs là -0,0572, -0,115 và -0,1654. Mọi tương quan đều âm: nghe ít đi thì dễ rời bỏ hơn. Một mô hình chỉ nhìn giá trị trung bình sẽ bỏ lỡ phần lớn tín hiệu; mô hình hồi quy đọc cả chuỗi nên thấy được sự thay đổi này.



Hình 2.8. Tương quan từng đặc trưng với nhân churn theo ba cách tóm tắt chuỗi 31 ngày

```

1 # a06/data_churn.py
2 def stratified_split(y, seed=SEED, fractions=(0.70, 0.15, 0.15)):
3     """Chỉ số train/val/test phân tầng theo nhãn (mỗi phần giữ đúng tỉ lệ churn)."""
4     rng = np.random.default_rng(seed)
5     parts = {"train": [], "val": [], "test": []}
6     for c in np.unique(y):
7         idx = rng.permutation(np.flatnonzero(y == c))
8         a = int(round(fractions[0] * len(idx)))
9         b = a + int(round(fractions[1] * len(idx)))
10        parts["train"].append(idx[:a])
11        parts["val"].append(idx[a:b])
12        parts["test"].append(idx[b:])
13    return {k: np.sort(np.concatenate(v)) for k, v in parts.items()}

```

Mã nguồn 17. Chia train/val/test phân tầng theo nhãn

Chương 3

Hiện thực và huấn luyện bằng PyTorch

Chương này dựng bốn mô hình Simple RNN, LSTM, GRU và BiLSTM bằng PyTorch [17], huấn luyện trên cả hai tập của Chương 2 và đánh giá trên tập test. Bốn mô hình chỉ khác nhau ở tầng hồi quy; phần còn lại (đầu dự báo, tối ưu, dữ liệu) giống hệt, để chênh lệch kết quả chỉ đến từ loại tế bào.

3.1 Một lớp cho bốn mô hình

Mô hình nhận tensor $(B, T, 8)$, cho qua tầng hồi quy 64 đơn vị, lấy vector tóm tắt h_T (hoặc $[\vec{h}_T; \overleftarrow{h}_1]$ với BiLSTM) rồi đưa qua đầu dự báo $\text{Linear}(\cdot, 32) \rightarrow \text{ReLU} \rightarrow \text{Linear}(32, 1)$. Đầu ra là một số: return đã chuẩn hoá với AMZN, logit churn với KKBBox. Cả bốn mô hình dùng chung một lớp; tham số cell chọn tầng hồi quy.

```
1 # a06/models_torch.py
2 class PyTorchRNN(nn.Module):
3     """cell ∈ {rnn, lstm, gru, bilstm}; đầu vào (B, T, D), đầu ra (B,)."""
4
5     def __init__(self, cell, n_features=8, hidden=HIDDEN, head=HEAD):
6         super().__init__()
7         layer = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU, "bilstm": nn.LSTM}[cell]
8         self.bidirectional = cell == "bilstm"
9         self.hidden = hidden
10        self.rnn = layer(n_features, hidden, batch_first=True, bidirectional=self.bidirectional)
11        width = hidden * (2 if self.bidirectional else 1)
12        self.head = nn.Sequential(nn.Linear(width, head), nn.ReLU(), nn.Linear(head, 1))
13
14    def forward(self, x):
15        out, _ = self.rnn(x) # (B, T, H) hoặc (B, T, 2H)
16        if self.bidirectional:
17            # chiều xuôi đọc xong ở bước cuối, chiều ngược đọc xong ở bước đầu: [h→_T; h←_1]
18            h = torch.cat([out[:, -1, :self.hidden], out[:, 0, self.hidden:]], dim=1)
19        else:
20            h = out[:, -1] # h_T tóm tắt cả chuỗi
21        return self.head(h).squeeze(-1)
```

Mã nguồn 18. Lớp chung cho Simple RNN, LSTM, GRU và BiLSTM

`nn.LSTM` và `nn.GRU` gọi thẳng nhân `cuDNN` khi chạy trên GPU, nên vòng lặp theo thời gian không nằm trong Python. Với BiLSTM, PyTorch trả về đầu ra hai chiều nối nhau ở mỗi bước; lớp trên cắt đúng $\vec{h}_T$ ở bước cuối và $\overleftarrow{h}_1$ ở bước đầu, đúng như cách nối đã kiểm chứng ở Chương 1.

3.2 Vòng lặp huấn luyện

Cấu hình giống nhau cho mọi mô hình: AdamW [18] với tốc độ học 10^{-3} và weight decay 10^{-4} , cắt chuẩn gradient về 1 (Chương 1), dừng sớm khi val loss không giảm sau một số epoch, và giữ lại trọng số của epoch có val loss nhỏ nhất. Bảng 3.1 ghi phần khác nhau giữa hai tập.

Bảng 3.1. Cấu hình huấn luyện

	AMZN (hồi quy)	KKBox (phân loại)
Hàm mất mát	MSE trên return chuẩn hoá	BCE trên logit, pos_weight = số âm / số dương
Lô, số epoch tối đa	64, 60	512, 60
Dừng sớm	8 epoch không cải thiện	4 epoch không cải thiện
Thiết bị	chọn bằng đo (Mục 3.2.1)	GPU
Seed	42	42

Toàn bộ dữ liệu được chuyển lên thiết bị một lần dưới dạng tensor; mỗi epoch chỉ xáo chỉ số rồi cắt lô, không dùng DataLoader. Với tập KKBox khoảng 100 MB, cách này bỏ được chi phí chép dữ liệu từ RAM sang GPU ở mỗi lô.

```

1 # a06/train_torch.py
2 def fit(model, data, cfg, device, max_epochs=None):
3     """Huấn Luyện một mô hình. data: dict X/y theo train/val (NumPy). Trả về model tốt nhất + history."""
4     torch.manual_seed(SEED)
5     Xtr = torch.as_tensor(data["X"]["train"], device=device)
6     ytr = torch.as_tensor(data["y"]["train"], device=device)
7     Xva = torch.as_tensor(data["X"]["val"], device=device)
8     yva = torch.as_tensor(data["y"]["val"], device=device)
9     model = model.to(device)
10    loss_fn = make_loss(cfg["task"], data["y"]["train"], device)
11    opt = torch.optim.AdamW(model.parameters(), lr=OPT["lr"], weight_decay=OPT["weight_decay"])
12    gen = torch.Generator(device="cpu").manual_seed(SEED)
13    hist = {"epoch": [], "train_loss": [], "val_loss": [], "epoch_s": []}
14    best, best_state, wait = float("inf"), None, 0
15    for epoch in range(1, (max_epochs or cfg["epochs"]) + 1):
16        sync(device)
17        t0 = time.perf_counter()
18        model.train()
19        perm = torch.randperm(len(Xtr), generator=gen).to(device)
20        total = 0.0
21        for i in range(0, len(Xtr), cfg["batch"]):
22            idx = perm[i:i + cfg["batch"]]
23            opt.zero_grad(set_to_none=True)
24            loss = loss_fn(model(Xtr[idx]), ytr[idx])
25            loss.backward()
26            nn.utils.clip_grad_norm_(model.parameters(), OPT["clip"])
27            opt.step()
28            total += loss.item() * len(idx)
29        sync(device)
30        elapsed = time.perf_counter() - t0
31        val = evaluate_loss(model, Xva, yva, loss_fn, min(cfg["batch"] * 8, 2048))
32        hist["epoch"].append(epoch)
33        hist["train_loss"].append(total / len(Xtr))
34        hist["val_loss"].append(val)
35        hist["epoch_s"].append(elapsed)
36        if val < best - 1e-6:
37            best, best_state, wait = val, copy.deepcopy(model.state_dict()), 0
38        else:
39            wait += 1
40            if wait >= cfg["patience"]:
41                break
42    model.load_state_dict(best_state)
43    hist["best_epoch"] = hist["epoch"][int(np.argmin(hist["val_loss"]))]
44    return model, hist

```

Mã nguồn 19. Vòng lặp huấn luyện chung: AdamW, clip gradient, dừng sớm theo val loss

Với KKBox, nhân dương chỉ chiếm khoảng 9%. Không cân bằng lại thì mô hình đạt loss thấp bằng cách đoán

“ở lại” cho tất cả. `pos_weight` nhân mất mát của mẫu dương lên khoảng 10 lần, tương đương cân bằng hai lớp trong hàm mất mát.

```
1 # a06/train_torch.py
2 def make_loss(task, y_train=None, device="cpu"):
3     """Hồi quy: MSE. Phân Loại: BCE trên Logit, pos_weight = số âm / số dương của train."""
4     if task == "reg":
5         return nn.MSELoss()
6     pos = float(y_train.sum())
7     pw = torch.tensor((len(y_train) - pos) / pos, device=device)
8     return nn.BCEWithLogitsLoss(pos_weight=pw)
```

Mã nguồn 20. Hàm mất mát cho hai loại bài toán

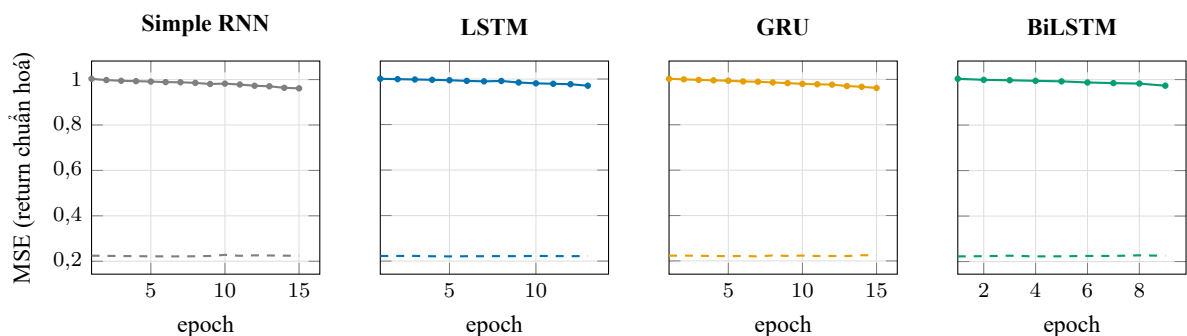
3.2.1 Chọn thiết bị bằng đo

Các bài trước cho thấy GPU không phải lúc nào cũng nhanh hơn với mô hình nhỏ và tập nhỏ. Vì vậy notebook 03 đo một epoch LSTM trên cả hai thiết bị (sau một lượt khởi động) rồi chọn bên nhanh hơn. Kết quả: CPU 0,652 s/epoch, GPU 0,2231 s/epoch, nên AMZN cũng huấn luyện trên GPU. Nhân cuDNN cho LSTM chạy cả 30 bước trong một lần gọi, còn CPU phải lặp từng bước.

```
1 # a06/train_torch.py
2 def pick_device(data, cfg):
3     """Đo 1 epoch của LSTM trên CPU và GPU, trả về thiết bị nhanh hơn (tập nhỏ có thể nhanh hơn trên CPU)."""
4     times = {}
5     for dev in ["cpu"] + (["cuda"] if torch.cuda.is_available() else []):
6         fit(build_torch("lstm"), data, cfg, dev, max_epochs=1) # Lượt làm nóng (cuDNN, bộ cấp phát)
7         _, h = fit(build_torch("lstm"), data, cfg, dev, max_epochs=1)
8         times[dev] = h["epoch_s"][0]
9     return min(times, key=times.get), times
```

Mã nguồn 21. Chọn CPU hay GPU bằng cách đo một epoch

3.3 Kết quả trên AMZN



Hình 3.1. Đường loss của bốn mô hình PyTorch trên AMZN (liền: train, đứt: val)

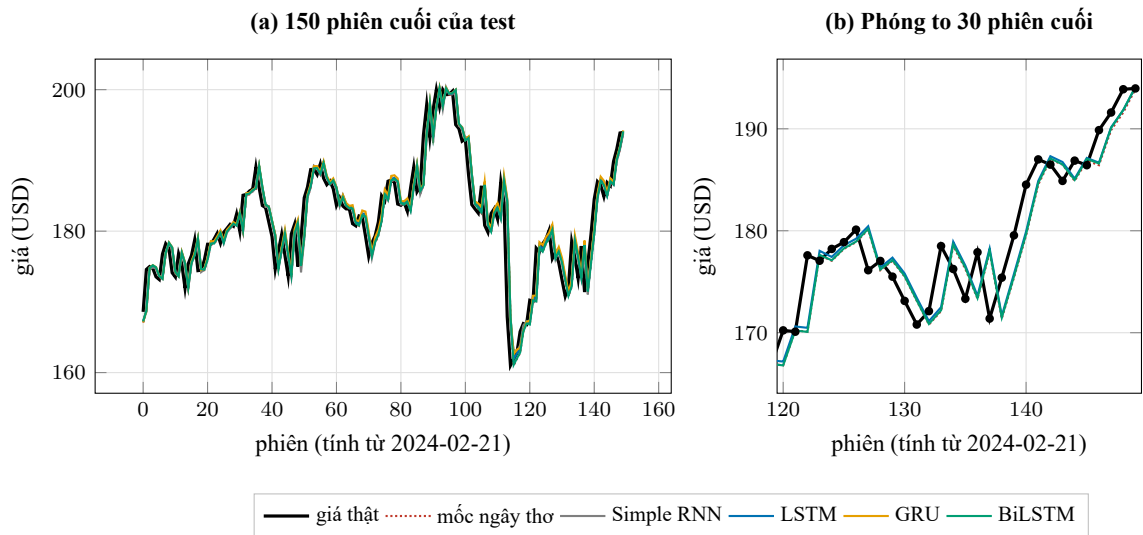
Hình 3.1 cho thấy train loss giảm chậm nhưng đều, còn val loss gần như không đổi ngay từ những epoch đầu. Val loss thấp hơn train loss nhiều vì nhân được chuẩn hoá theo độ lệch chuẩn của train, trong khi return giai đoạn val biến động ít hơn hẳn (Chương 2); so hai đường với nhau thì không có ý nghĩa, chỉ xu hướng của từng đường là có. BiLSTM có val loss nhỏ nhất ở epoch 1, tức là mọi epoch sau đó chỉ làm mô hình khớp thêm nhiều của tập train. Các mô hình còn lại đạt val loss nhỏ nhất ở epoch 5 đến 7.

Bảng 3.2. Bốn mô hình PyTorch trên tập test AMZN (2020-09-01 đến 2024-09-24)

Mô hình	RMSE	MAE	R^2 giá	R^2 return (%)	Đúng chiều (%)	Tham số	Epoch tốt/chạy	s/epoch
Simple RNN	3,137	2,290	0,9880	-1,48	48,43	6.849	7/15	0,25
LSTM	3,131	2,275	0,9880	-0,63	51,47	21.057	5/13	0,21
GRU	3,141	2,286	0,9879	-1,36	50,59	16.321	7/15	0,17
BiLSTM	3,129	2,278	0,9880	-0,19	50,88	42.049	1/9	0,27
Mốc ngày thơ $\hat{P}_{t+1} = P_t$	3,124	2,274	0,9881	-0,00	—	0	—	—

RMSE, MAE tính bằng USD trên tập test; chữ đậm: nhỏ nhất.

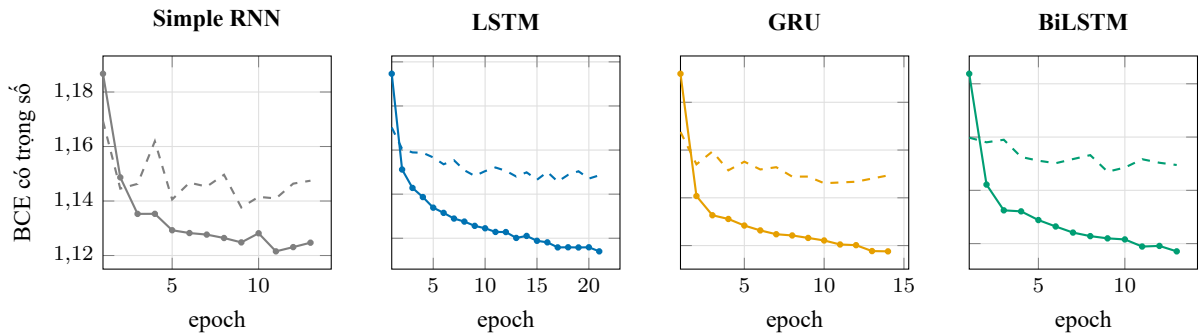
Bảng 3.2 là kết quả trung thực nhất của bài, và nó không đẹp. RMSE của bốn mô hình nằm trong khoảng 3,13 đến 3,14 USD, trong khi mốc ngày thơ “giá ngày mai bằng giá hôm nay” đạt 3,12 USD. Không mô hình nào thắng mốc ngày thơ. Cột R^2 trên giá cho 0,988 với mọi mô hình và với cả mốc ngày thơ, nên con số rất cao này không nói gì: giá hôm nay đã giải thích gần hết phương sai của giá ngày mai. Cột R^2 trên return mới đo khả năng dự báo thật, và nó âm với cả bốn mô hình. Tỷ lệ dự báo đúng chiều tăng hay giảm dao động quanh 50 %, từ 48,43 % đến 51,47 %.



Hình 3.2. Giá thật, mốc ngày thơ và dự báo của bốn mô hình trên các phiên cuối của tập test

Hình 3.2 cho thấy nhìn bằng mắt thì dự báo “bám” giá thật rất sát. Phóng to 30 phiên cuối (ô b) thì thấy rõ lý do: đường dự báo gần như trùng mốc ngày thơ, tức là đường giá thật dịch sang phải một phiên. Mô hình học được rằng return ngày mai có trung bình gần 0, và dự báo gần 0 là lựa chọn có sai số bình phương nhỏ nhất khi return không có tự tương quan (Chương 2). Chương 4 bàn tiếp giới hạn này.

3.4 Kết quả trên KKBox



Hình 3.3. Đường loss của bốn mô hình PyTorch trên KKBox (liền: train, đứt: val)

KKBox là bài toán khác hẳn. Train loss và val loss giảm cùng nhau trong nhiều epoch (Hình 3.3); LSTM đạt val loss tốt nhất ở epoch 17/21. Mô hình học được tín hiệu thật từ chuỗi hành vi.

Ngưỡng phân loại không cố định ở 0,5. Với mỗi mô hình, notebook chọn ngưỡng làm F1 lớn nhất trên *validation*, rồi áp nguyên ngưỡng đó cho test. Chọn ngưỡng trên test sẽ làm kết quả test đẹp hơn thực tế.

```
1 # a06/bench.py
2 def best_threshold(y, prob):
3     """Ngưỡng làm F1 Lớn nhất, chọn trên VALIDATION rồi mới áp cho test."""
4     prec, rec, thr = precision_recall_curve(y, prob)
5     f1 = 2 * prec[:-1] * rec[:-1] / np.maximum(prec[:-1] + rec[:-1], 1e-12)
6     return float(thr[int(np.argmax(f1))])
```

Mã nguồn 22. Chọn ngưỡng theo F1 trên validation

Bảng 3.3. Bốn mô hình PyTorch trên tập test KKBox (15.000 người dùng)

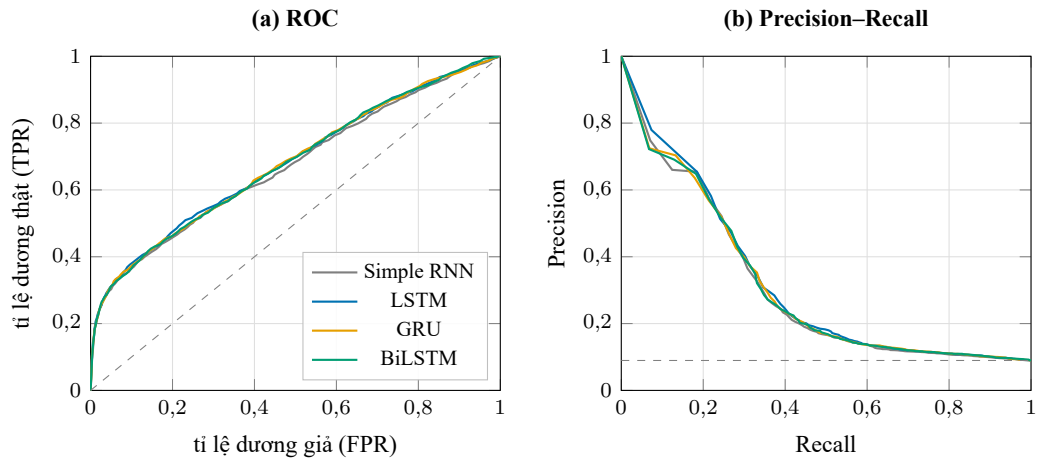
Mô hình	Acc	Precision	Recall	F1	ROC-AUC	PR-AUC	Ngưỡng	Epoch tốt/chạy	s/epoch
Simple RNN	90,03	41,80	28,65	34,00	0,6720	0,2923	0,695	9/13	0,46
LSTM	89,33	38,26	31,03	34,26	0,6819	0,3102	0,657	17/21	0,57
GRU	90,36	43,91	27,38	33,73	0,6802	0,3004	0,761	10/14	0,62
BiLSTM	89,07	37,14	31,70	34,20	0,6801	0,2975	0,684	9/13	1,11

Acc, Precision, Recall, F1 theo %, tại ngưỡng chọn trên validation; chữ đậm: tốt nhất.

Ba mô hình có cổng (LSTM, GRU, BiLSTM) đạt ROC-AUC từ 0,6802 đến 0,6819, cao hơn Simple RNN (0,672). Khoảng cách không lớn vì chuỗi chỉ dài 31 bước, chưa đủ dài để gradient của Simple RNN triệt tiêu hoàn toàn. LSTM có F1 cao nhất, 34,26 %, với Precision 38,26 % và Recall 31,03 %: cứ ba người bị báo churn thì khoảng một người churn thật, và mô hình tìm ra khoảng một phần ba số người churn.

	Simple RNN		LSTM		GRU		BiLSTM	
thực tế	ở lại	churn	ở lại	churn	ở lại	churn	ở lại	churn
	13.120	536	12.983	673	13.186	470	12.935	721
đưa ra dự đoán	959	385	927	417	976	368	918	426

Hình 3.4. Ma trận nhầm lẫn trên test KKBox; màu ô theo tỉ lệ trong hàng



Hình 3.5. Đường ROC và Precision-Recall trên test KKBox; vạch đứt là mô hình đoán ngẫu nhiên

Các con số này thấp so với bảng xếp hạng của cuộc thi, và có lý do. Các đội mạnh nhất dùng thêm lịch sử giao dịch (phương thức thanh toán, tự động gia hạn, ngày hết hạn), là những đặc trưng dự báo churn trực tiếp. Bài này chỉ dùng log nghe nhạc, vì trọng tâm của bài là mô hình hồi quy trên dữ liệu chuỗi. Hình 3.5 cho thấy phần tín hiệu mà chuỗi hành vi mang lại là có thật: đường PR nằm cao hơn nhiều so với mốc ngẫu nhiên 8,96 %, nhưng ROC-AUC dừng ở khoảng 0,68. Phần tín hiệu còn lại không nằm trong việc người dùng nghe nhạc thế nào.

Chương 4

Keras và đối chuẩn hai framework

Chương này viết lại bốn mô hình bằng Keras [19] (backend TensorFlow [20]), huấn luyện trên cùng dữ liệu, rồi so 16 mô hình theo chất lượng, kích thước và tốc độ. Mọi chỉ số của cả hai framework được tính lại bằng cùng một hàm trong `a06/bench.py`, từ dự báo đã lưu, để phép so không lệch vì cách tính.

4.1 Bốn mô hình bằng Keras

Kiến trúc giữ đúng như bản PyTorch: đầu vào `Input(T, 8)`, một tầng hồi quy 64 đơn vị (một trong `SimpleRNN`, `LSTM`, `GRU` và `Bidirectional(LSTM)`), rồi hai tầng `Dense(32, relu)` và `Dense(1)`. Tối ưu cũng giống: `AdamW` với tốc độ học 10^{-3} , `weight decay` 10^{-4} , `clipnorm=1.0`.

```
1 # a06/models_keras.py
2 def build_keras(cell, task, T, n_features=8):
3     """Input(T, D) -> tầng hồi quy 64 -> Dense(32, relu) -> Dense(1) (logit nếu phân loại)."""
4     rnn = {"rnn": lambda: layers.SimpleRNN(HIDDEN),
5            "lstm": lambda: layers.LSTM(HIDDEN),
6            "gru": lambda: layers.GRU(HIDDEN),
7            "bilstm": lambda: layers.Bidirectional(layers.LSTM(HIDDEN))}[cell]
8     model = keras.Sequential([layers.Input((T, n_features)), rnn(),
9                               layers.Dense(HEAD, activation="relu"), layers.Dense(1)],
10                              name=f"keras_{cell}")
11     loss = (keras.losses.MeanSquaredError() if task == "reg"
12             else keras.losses.BinaryCrossentropy(from_logits=True))
13     model.compile(optimizer=keras.optimizers.AdamW(learning_rate=OPT["lr"], weight_decay=OPT["weight_decay"],
14                                                    clipnorm=OPT["clip"]), loss=loss)
15     return model
```

Mã nguồn 23. Bốn mô hình Keras dựng từ một hàm

Có một khác biệt nhỏ về clipping. `clipnorm` của Keras cắt chuẩn của từng tensor gradient, còn `clip_grad_norm` của PyTorch cắt theo chuẩn gộp của mọi gradient. Với ngưỡng 1 và mô hình nhỏ như ở đây, cả hai hiếm khi được kích hoạt sau vài epoch đầu, nên bài giữ mặc định của mỗi framework.

Keras không có tham số `pos_weight`. Cách tương đương là trọng số mẫu: mẫu dương nhận trọng số bằng số âm chia số dương, mẫu âm nhận 1. Trọng số được truyền cho cả tập train lẫn tập validation, để `val loss` mà `EarlyStopping` theo dõi đo đúng đại lượng như `val loss` của PyTorch.

```
1 # a06/models_keras.py
2 def sample_weights(task, y, y_train):
3     """Trọng số mẫu tương đương pos_weight của BCEWithLogitsLoss: dương nặng (âm/dương) lần."""
4     if task == "reg":
5         return None
6     pw = (len(y_train) - y_train.sum()) / y_train.sum()
7     return np.where(y > 0.5, pw, 1.0).astype(np.float32)
```

Mã nguồn 24. Trọng số mẫu thay cho `pos_weight`

```

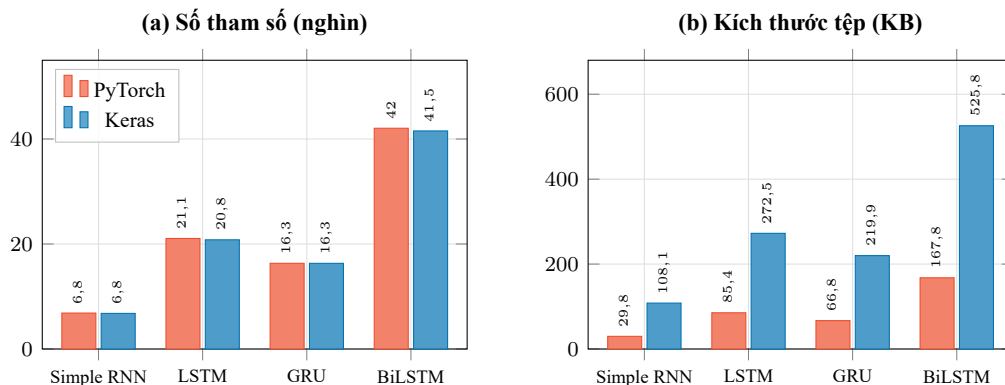
1 # a06/models_keras.py
2 def fit_keras(cell, data, cfg):
3     keras.utils.set_random_seed(SEED)
4     Xtr, ytr, Xva, yva = data["X"]["train"], data["y"]["train"], data["X"]["val"], data["y"]["val"]
5     model = build_keras(cell, cfg["task"], Xtr.shape[1], Xtr.shape[2])
6     sw_tr = sample_weights(cfg["task"], ytr, ytr)
7     sw_va = sample_weights(cfg["task"], yva, ytr)
8     val = (Xva, yva) if sw_va is None else (Xva, yva, sw_va)
9     timer = EpochTimer()
10    stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=cfg["patience"],
11                                       restore_best_weights=True)
12    h = model.fit(Xtr, ytr, sample_weight=sw_tr, validation_data=val, epochs=cfg["epochs"],
13                batch_size=cfg["batch"], shuffle=True, callbacks=[stop, timer], verbose=0)
14    n = len(h.history["loss"])
15    hist = {"epoch": list(range(1, n + 1)), "train_loss": h.history["loss"],
16           "val_loss": h.history["val_loss"], "epoch_s": timer.times,
17           "best_epoch": int(np.argmin(h.history["val_loss"])) + 1}
18    return model, hist

```

Mã nguồn 25. Huấn luyện một mô hình Keras với cùng lô và cùng dừng sớm

4.1.1 Chênh lệch số tham số

Cùng kiến trúc nhưng số tham số không bằng nhau (Hình 4.1a). PyTorch lưu hai vector bias cho mỗi khối cổng (b_{ih} và b_{hh}), Keras chỉ lưu một. Vì vậy Keras ít hơn đúng $H = 64$ tham số với Simple RNN (-64), $4H$ với LSTM (-256) và $8H$ với BiLSTM (-512). GRU là ngoại lệ: Keras mặc định `reset_after=True`, áp cổng reset sau phép nhân ma trận giống PyTorch, nên cần hai vector bias và số tham số bằng nhau (16.321). Bài giữ nguyên chênh lệch này thay vì ép bằng, vì nó không đổi khả năng biểu diễn: tổng hai bias chỉ là một bias.

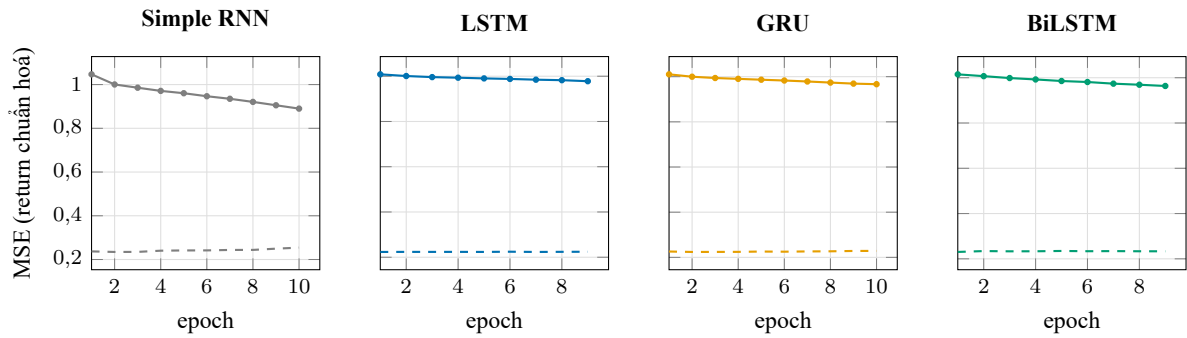


Hình 4.1. Số tham số và kích thước tệp mô hình của hai framework

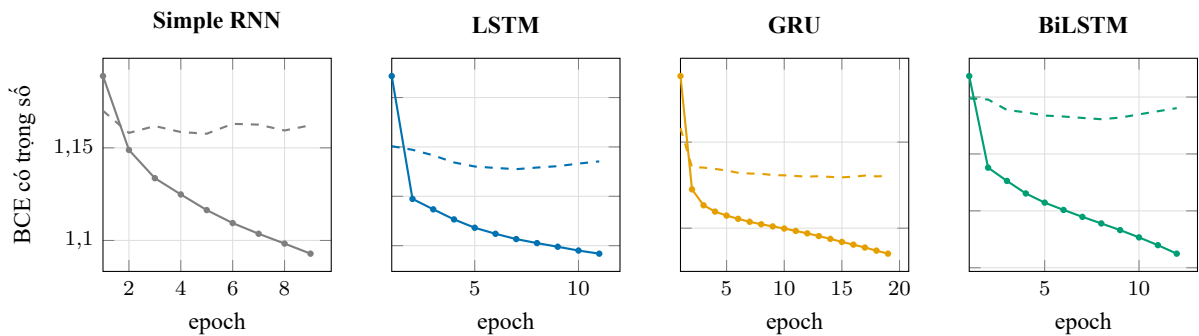
Kích thước tệp chênh lệch nhiều hơn số tham số. Tệp `.pth` chỉ chứa `state_dict`, còn tệp `.keras` mặc định lưu thêm cấu hình mô hình và trạng thái của optimizer (hai moment của AdamW cho mỗi trọng số), nên nặng gấp khoảng ba lần: 272,46 KB so với 85,36 KB với LSTM. Nhờ vậy tệp Keras huấn luyện tiếp được ngay, nhưng khi chỉ cần phục vụ thì phần optimizer là thừa.

4.2 Huấn luyện song song trên hai thiết bị

TensorFlow từ bản 2.11 không còn hỗ trợ GPU trên Windows, nên Keras chạy trên CPU (16 luồng) trong một tiến trình riêng, cùng lúc với notebook PyTorch dùng GPU. Hai bên không phải chờ nhau, và CPU 32 luồng của máy được chia đôi cho hai tiến trình. Hình 4.2 và 4.3 là đường loss của bốn mô hình Keras; hình dạng giống bản PyTorch ở Chương 3: trên AMZN val loss đứng yên ngay từ đầu, trên KKBox cả hai đường cùng giảm.



Hình 4.2. Đường loss của bốn mô hình Keras trên AMZN (liền: train, đứt: val)



Hình 4.3. Đường loss của bốn mô hình Keras trên KKBBox (liền: train, đứt: val)

Thời gian huấn luyện phản ánh chủ yếu thiết bị, không phản ánh framework. Trên KKBBox, một epoch BiLSTM mất 17,55 giây với Keras trên CPU và 1,11 giây với PyTorch trên GPU. Vì vậy phần so tốc độ bên dưới đo suy luận trên cùng một thiết bị.

4.3 Đo độ trễ suy luận

Để so công bằng, độ trễ của cả 16 mô hình được đo trên CPU với đúng một luồng. Mỗi mô hình chạy khởi động 20 lần, rồi đo 200 lần với lô một mẫu và lấy trung vị (độ trễ), và đo 30 lần với lô 256 mẫu (thông lượng). Mô hình Keras chạy qua `tf.function`, tức đồ thị đã biên dịch, là cách dùng khi phục vụ thật; gọi `model(x)` ở chế độ eager chậm hơn khoảng một trăm lần vì vòng lặp theo thời gian chạy từng phép tính một.

```
1 # a06/bench.py
2 def _median_time(fn, x, warmup=20, repeat=200):
3     for _ in range(warmup):
4         fn(x)
5     ts = []
6     for _ in range(repeat):
7         t0 = time.perf_counter()
8         fn(x)
9         ts.append(time.perf_counter() - t0)
10    return float(np.median(ts))
```

Mã nguồn 26. Đo thời gian: khởi động rồi lấy trung vị

```

1 # a06/bench.py
2 def latency(fn, sample, batch_big=256):
3     """fn nhận mảng (B, T, D). Trả về ms/mẫu với lô 1 và mẫu/giây với lô 256 (trung vị)."""
4     x1 = sample[:1]
5     xb = np.repeat(sample[:1], batch_big, axis=0) if len(sample) < batch_big else sample[:batch_big]
6     t1 = _median_time(fn, x1)
7     tb = _median_time(fn, xb, warmup=5, repeat=30)
8     return {"latency_ms": 1000 * t1, "throughput": batch_big / tb}

```

Mã nguồn 27. Độ trễ với lô 1 và thông lượng với lô 256

Khi viết notebook này, lần đo đầu tiên cho PyTorch chậm bất thường, 1,5 đến 5 ms mỗi mẫu. Nguyên nhân là mô hình PyTorch và Keras được đo xen kẽ nhau: sau mỗi lần gọi, các luồng của TensorFlow tiếp tục quay chờ việc một lúc và giành mất lõi CPU duy nhất của PyTorch. Đo hết PyTorch rồi mới đo Keras thì kết quả ổn định. Đây là một ví dụ nhỏ cho thấy đối chuẩn hiệu năng dễ sai như thế nào nếu không kiểm soát môi trường.

4.4 Bảng đối chuẩn 16 mô hình

Bảng 4.1. Đối chuẩn trên AMZN: 8 mô hình và mốc ngày thơ

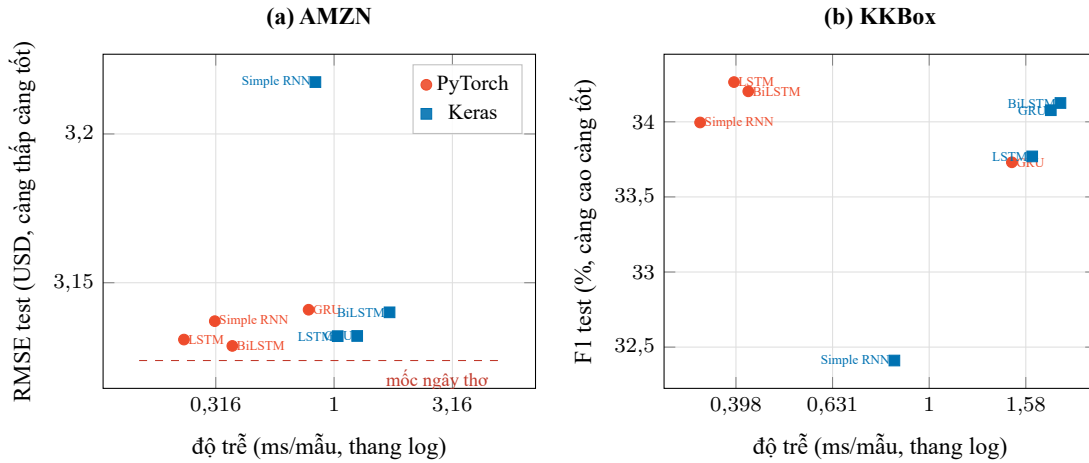
Framework	Mô hình	RMSE	MAE	R^2 ret. (%)	Tham số	KB	s/epoch	ms/mẫu	mẫu/s
PyTorch	Simple RNN	3,137	2,290	-1,48	6.849	29,8	0,25 (GPU)	0,313	83.344
	LSTM	3,131	2,275	-0,63	21.057	85,4	0,21 (GPU)	0,232	53.314
	GRU	3,141	2,286	-1,36	16.321	66,8	0,17 (GPU)	0,781	37.275
	BiLSTM	3,129	2,278	-0,19	42.049	167,8	0,27 (GPU)	0,371	26.223
Keras	Simple RNN	3,217	2,358	-4,77	6.785	108,1	0,82 (CPU)	0,834	81.669
	LSTM	3,132	2,282	-0,37	20.801	272,5	1,16 (CPU)	1,036	28.993
	GRU	3,132	2,275	-0,41	16.321	219,9	1,29 (CPU)	1,252	35.358
	BiLSTM	3,140	2,283	-1,36	41.537	525,8	2,08 (CPU)	1,716	14.714
Mốc ngày thơ		3,124	2,274	-0,00	0	-	-	-	-

Độ trễ (lô 1, trung vị 200 lần) và thông lượng (lô 256) đo trên CPU, 1 luồng, cả hai framework. s/epoch đo lúc huấn luyện, trên thiết bị ghi trong ngoặc.

Bảng 4.2. Đối chuẩn trên KKBox: 8 mô hình

Framework	Mô hình	F1 (%)	Recall (%)	ROC-AUC	Tham số	KB	s/epoch	ms/mẫu	mẫu/s
PyTorch	Simple RNN	34,00	28,65	0,6720	6.849	29,9	0,46 (GPU)	0,336	48.698
	LSTM	34,26	31,03	0,6819	21.057	85,4	0,57 (GPU)	0,394	48.671
	GRU	33,73	27,38	0,6802	16.321	66,9	0,62 (GPU)	1,483	22.345
	BiLSTM	34,20	31,70	0,6801	42.049	167,8	1,11 (GPU)	0,422	24.380
Keras	Simple RNN	32,41	27,01	0,6616	6.785	108,2	2,60 (CPU)	0,847	113.624
	LSTM	33,77	27,83	0,6787	20.801	272,5	9,87 (CPU)	1,634	17.626
	GRU	34,08	29,54	0,6797	16.321	220,0	8,26 (CPU)	1,785	35.230
	BiLSTM	34,13	29,39	0,6776	41.537	525,9	17,55 (CPU)	1,870	14.364

Độ trễ (lô 1, trung vị 200 lần) và thông lượng (lô 256) đo trên CPU, 1 luồng, cả hai framework. s/epoch đo lúc huấn luyện, trên thiết bị ghi trong ngoặc.



Hình 4.4. Chất lượng theo độ trễ suy luận của 16 mô hình

Bảng 4.1, Bảng 4.2 và Hình 4.4 cho bốn kết luận.

Chất lượng gần như không phụ thuộc framework. Trên KKBBox, F1 của cùng một kiến trúc ở hai framework chênh nhau từ $-1,58$ (Simple RNN) đến $+0,35$ (GRU) điểm phần trăm. Trên AMZN, RMSE chênh không quá 0,0113 USD, trừ Simple RNN của Keras kém hơn 0,0803 USD vì dừng ở epoch 2. Mỗi mô hình chỉ chạy một seed, nên những chênh lệch cỡ này nằm trong mức dao động của khởi tạo ngẫu nhiên, không đủ để nói framework nào học tốt hơn.

PyTorch nhanh hơn với từng mẫu một. Độ trễ trung vị của 8 mô hình PyTorch là 0,383 ms, của Keras là 1,44 ms. Với lô một mẫu, chi phí gọi hàm của framework chiếm phần lớn thời gian, và lời gọi eager của PyTorch nhẹ hơn lời gọi `tf.function`. BiLSTM chênh nhiều nhất: Keras chậm hơn 4,62 lần.

Với lô lớn, khoảng cách hẹp lại và có chỗ đảo chiều. Ở lô 256, Keras Simple RNN xử lý 113.624 mẫu/giây trên KKBBox, nhiều hơn PyTorch (48.698); Keras GRU trên KKBBox cũng nhanh hơn (35.230 so với 22.345). Với LSTM và BiLSTM, PyTorch vẫn nhanh hơn, từ 1,7 đến 2,76 lần. Chọn framework theo tốc độ cần biết trước ứng dụng phục vụ từng yêu cầu một hay theo lô.

Mô hình “tốt nhất” khác nhau tùy tiêu chí. Theo val loss (tiêu chí dùng để chọn mô hình cho Web App), mô hình tốt nhất trên AMZN là GRU (PyTorch) và LSTM (Keras), trên KKBBox là LSTM (PyTorch) và GRU (Keras). Chỉ số test tốt nhất lại thuộc về BiLSTM (PyTorch) trên AMZN và LSTM (PyTorch) trên KKBBox. Khi các mô hình gần bằng nhau như vậy, thứ hạng trên test gần như là ngẫu nhiên, và mô hình rẻ nhất đủ tốt là lựa chọn hợp lý.

4.5 Giới hạn của mô hình dự báo giá

Trên AMZN, không mô hình nào trong 16 mô hình thắng mốc ngày thơ (dòng cuối Bảng 4.1). Đây không phải lỗi cài đặt. Nó đến từ ba điều đã thấy trong dữ liệu:

- **Return gần như không có tự tương quan** (Chương 2): quá khứ 30 phiên chứa rất ít thông tin về return ngày mai. Dự báo tốt nhất theo sai số bình phương khi đó là một hằng số gần 0, đúng là điều mô hình học được (val loss đứng yên từ epoch đầu).
- **Trễ pha tại điểm đảo chiều:** vì $\hat{r} \approx 0$, dự báo $\hat{P}_{t+1} \approx P_t$ luôn đi sau giá thật một phiên (Hình 3.2b). Khi giá đổi chiều, mô hình chỉ “thấy” sau khi việc đã xảy ra.
- **Phân phối đổi theo thời gian:** độ lệch chuẩn của return là 0,0391 trên train nhưng chỉ 0,0225 trên test (Chương 2), và những phiên lớn nhất đến từ sự kiện ngoài dữ liệu (báo cáo tài chính, tin vĩ mô) mà tám đặc trưng kỹ thuật không chứa.

Chỉ số R^2 trên giá (khoảng 0,99) và biểu đồ “đường dự báo bám sát đường thật” là hai cách trình bày dễ làm kết quả trông tốt hơn thực tế. Với chuỗi giá, cần so với mốc ngày thơ và đo trên return.

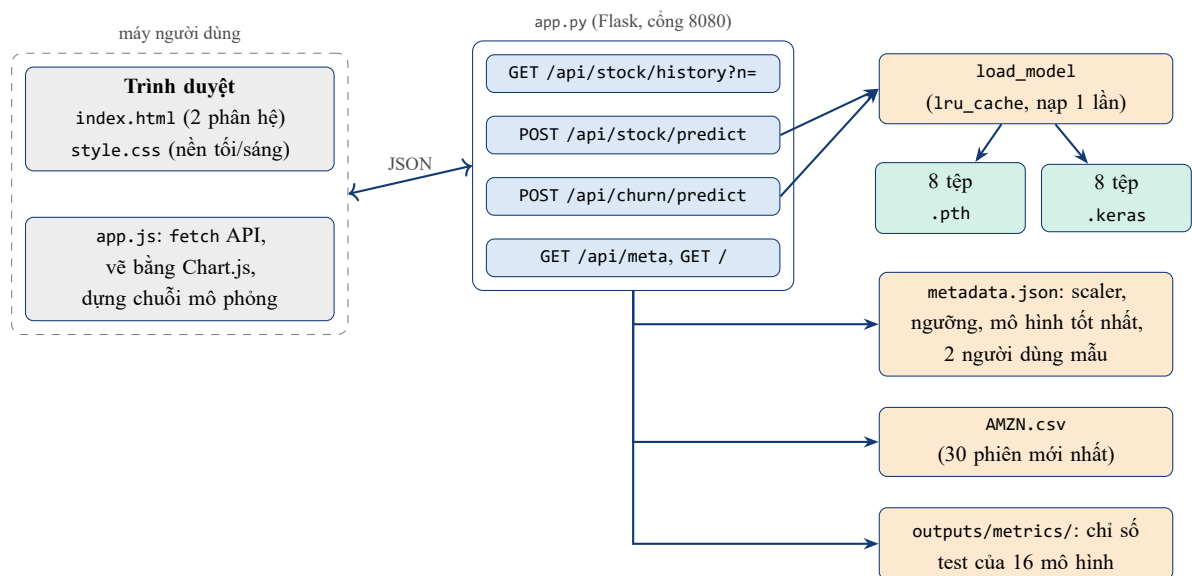
Chương 5

Triển khai Web App

Chương này đóng gói các mô hình thành một Web App chạy cục bộ. Ứng dụng có hai phân hệ ứng với hai tập dữ liệu: dự báo giá đóng cửa AMZN của phiên kế tiếp, và dự báo xác suất churn của một người dùng KKBox từ chuỗi 31 ngày nghe nhạc. Người dùng chọn framework (PyTorch hoặc Keras) và một trong bốn mô hình; mặc định là mô hình có val loss nhỏ nhất (Chương 4).

5.1 Kiến trúc

Máy chủ là Flask, trả JSON qua bốn nhóm endpoint; giao diện là một trang HTML với hai phân hệ, gọi API bằng fetch và vẽ biểu đồ bằng Chart.js (Hình 5.1). Ứng dụng không cần dữ liệu gốc của KKBox để chạy. Mô hình và tham số tiền xử lý nằm trong thư mục `models/`: 16 tệp mô hình và `metadata.json` do notebook 06 ghi, gồm thống kê chuẩn hoá của hai tập, ngưỡng phân loại chọn trên validation, tên mô hình tốt nhất và chuỗi hành vi của hai người dùng mẫu. Chỉ số test của 16 mô hình mà giao diện hiện trên thẻ mô hình được đọc từ `outputs/metrics/`, đúng các tệp mà Chương 3 và 4 dùng để lập bảng, nên con số trên giao diện và trong báo cáo là một.



Hình 5.1. Kiến trúc Web App

Bảng 5.1 liệt kê các endpoint.

Bảng 5.1. REST API của Web App

Phương thức	Đường dẫn	Vào → ra
GET	/api/stock/history?n=	$n \in \{60, 100, 250\} \rightarrow n$ phiên gần nhất: ngày, Close, MA20, RSI-14 và giá mở/cao/thấp/khối lượng của phiên cuối
POST	/api/stock/predict	{framework, model} $\rightarrow$ giá dự báo, % thay đổi, RSI-14 và vùng RSI, ghi chú rủi ro, độ trễ
POST	/api/churn/predict	{preset} hoặc {sequence} (31×8), framework, model $\rightarrow$ xác suất, mức rủi ro, ngưỡng, xu hướng nghe, khuyến nghị, độ trễ
GET	/api/meta	$\rightarrow$ danh sách mô hình, mô hình tốt nhất, thẻ chỉ số test của 16 mô hình, thống kê hai tập, chuỗi của hai người dùng mẫu
GET	/	$\rightarrow$ trang giao diện

5.2 Nạp mô hình

Mỗi mô hình chỉ được nạp một lần, ở lần gọi đầu tiên, rồi giữ trong bộ nhớ nhờ `lru_cache`. Hàm nạp trả về một hàm dự báo cùng giao diện cho cả hai framework, nên phần xử lý yêu cầu không cần biết mô hình đến từ đâu. Mô hình Keras chạy qua `tf.function` như lúc đo độ trễ ở Chương 4, và mỗi mô hình được chạy thử một lần ngay khi nạp để độ trễ trả về cho người dùng là độ trễ lúc ổn định. TensorFlow chỉ được import khi có yêu cầu dùng mô hình Keras.

```

1 # app/app.py
2 @lru_cache(maxsize=None)
3 def load_model(dataset, framework, cell):
4     """Nạp một lần rồi giữ trong bộ nhớ. Trả về hàm f(X: (B, T, D) float32) -> đầu ra thô (B,).
5
6     Chạy thử một lần ngay khi nạp (Keras trace đồ thị ở lần gọi đầu) để độ trễ trả về là của lúc chạy ổn định.
7     """
8     info = META["models"][f"{dataset}_{framework}_{cell}"]
9     path = MODELS / info["file"]
10    if framework == "torch":
11        model = build_torch(cell)
12        model.load_state_dict(torch.load(path, map_location="cpu"))
13        model.eval()
14
15    def run(X):
16        with torch.no_grad():
17            return model(torch.as_tensor(X)).numpy()
18    else:
19        import keras # chỉ nạp TensorFlow khi thật sự cần mô hình Keras
20        import tensorflow as tf
21        model = keras.models.load_model(path)
22        graph = tf.function(lambda X: model(X, training=False), reduce_retracing=True) # eager chậm hơn ~100 lần
23
24    def run(X):
25        return graph(X).numpy().ravel()
26    run(np.zeros((1, META["seq_len"][dataset], 8), dtype=np.float32))
27    return run

```

Mã nguồn 28. Nạp mô hình một lần, cùng giao diện cho PyTorch và Keras

5.3 Hai phân hệ dự báo

Phân hệ giá lấy 30 phiên mới nhất trong AMZN.csv, áp dụng phép biến đổi của Chương 2 (giá tương đối, log Volume, z-score bằng thống kê của train), đưa qua mô hình rồi quy đổi return dự báo ra giá USD. Hàm latest_window dùng chung hai hàm biến đổi với make_windows của lúc huấn luyện, nên dữ liệu phục vụ không thể lệch khỏi dữ liệu huấn luyện.

```

1 # app/app.py
2 @app.post("/api/stock/predict")
3 def stock_predict():
4     body = request.get_json(silent=True) or {}
5     try:
6         framework, cell = pick("amzn", body)
7     except ValueError as e:
8         return jsonify({"error": str(e)}), 400
9     w, close, last = latest_window(STOCK_DF)
10    X = ((w - np.array(S_SCALER["mean"])) / np.array(S_SCALER["std"])).astype(np.float32)[None]
11    z, ms = timed(load_model("amzn", framework, cell), X)
12    r_hat = float(z[0]) * S_SCALER["y_std"] + S_SCALER["y_mean"] # return đã chuẩn hoá -> Log-return
13    pred = close * float(np.exp(r_hat))
14    rsi = float(STOCK_DF.RSI14.iloc[-1])
15    return jsonify({"framework": framework, "model": cell, "model_label": CELL_LABELS[cell],
16                  "last_date": last.strftime("%Y-%m-%d"), "last_close": round(close, 2),
17                  "pred_close": round(pred, 2), "change_pct": round(100 * (pred / close - 1), 3),
18                  "rsi14": round(rsi, 2), "rsi_zone": rsi_zone(rsi), "note": STOCK_NOTE,
19                  "latency_ms": round(ms, 3)})

```

Mã nguồn 29. Endpoint dự báo giá phiên kế tiếp

Kèm theo giá dự báo, phân hệ trả về RSI-14 của phiên mới nhất và vùng của nó theo ngưỡng 70/30 của Wilder [12] (quá mua, trung tính, quá bán). Mỗi dự báo cũng mang một ghi chú cố định: mô hình không thắng được mốc ngày thơ trên tập test (Chương 4), nên con số chỉ để minh hoạ, không phải khuyến nghị đầu tư. Giao diện hiện ghi chú này ngay dưới kết quả, để người dùng không đọc dự báo nhiều hơn những gì nó nói được.

Phân hệ churn nhận hoặc tên một người dùng mẫu, hoặc một ma trận 31×8 bất kỳ. Đầu vào sai shape bị từ chối với mã 400 và thông báo lỗi rõ ràng, không đưa vào mô hình. Xác suất được so với ngưỡng của đúng mô hình đó để xếp mức rủi ro: cao khi vượt ngưỡng, trung bình khi vượt nửa ngưỡng, còn lại là thấp.

```

1 # app/app.py
2 @app.post("/api/churn/predict")
3 def churn_predict():
4     body = request.get_json(silent=True) or {}
5     try:
6         framework, cell = pick("kkbox", body)
7         if "preset" in body:
8             seq = np.array(META["presets"][body["preset"]]["sequence"], dtype=np.float64)
9         else:
10            seq = np.array(body["sequence"], dtype=np.float64)
11            if seq.shape != (31, 8):
12                raise ValueError(f"sequence phải có shape (31, 8), nhận {seq.shape}")
13    except KeyError as e:
14        return jsonify({"error": f"thiếu hoặc sai khóa {e}"}), 400
15    except ValueError as e:
16        return jsonify({"error": str(e)}), 400
17    X = ((seq - np.array(C_SCALER["mean"])) / np.array(C_SCALER["std"])).astype(np.float32)[None]
18    z, ms = timed(load_model("kkbox", framework, cell), X)
19    prob = float(1 / (1 + np.exp(-z[0])))
20    thr = META["thresholds"][f"kkbox_{framework}_{cell}"]
21    risk = "cao" if prob >= thr else "trung bình" if prob >= thr / 2 else "thấp"
22    return jsonify({"framework": framework, "model": cell, "model_label": CELL_LABELS[cell],
23                  "prob": round(prob, 4), "threshold": round(thr, 4), "risk": risk,
24                  "advice": ADVICE[risk], "trend": listening_trend(seq),
25                  "latency_ms": round(ms, 3), "sequence": seq.round(4).tolist()})

```

Mã nguồn 30. Endpoint dự báo churn

Để kết quả dùng được cho bộ phận chăm sóc khách hàng, phân hệ trả thêm hai thông tin. Thứ nhất là *xu hướng nghe*: $\log(1 + \text{giây nghe})$ trung bình 7 ngày cuối trừ 7 ngày đầu, chính là tóm tắt có tương quan mạnh nhất với nhân churn ở Chương 2. Thứ hai là một *khuyến nghị can thiệp* theo mức rủi ro: rủi ro cao thì liên hệ trước ngày hết hạn gói và tặng ưu đãi gia hạn, rủi ro trung bình thì gửi nhắc gia hạn và theo dõi thêm, rủi ro thấp thì chăm sóc bình thường.

```

1 # app/app.py
2 def listening_trend(seq):
3     """Xu hướng nghe: Log(1+giây) trung bình 7 ngày cuối trừ 7 ngày đầu (Chương 2: đây là tín hiệu mạnh nhất)."""
4     secs = seq[:, META["kkbox_features"].index("total_secs")]
5     return round(float(secs[-7:].mean() - secs[:7].mean()), 4)

```

Mã nguồn 31. Xu hướng nghe trả về cùng xác suất churn

5.4 Thiết kế giao diện

Giao diện nhắm tới hai kiểu người dùng: người phân tích muốn xem mô hình nói gì và đáng tin tới đâu, và nhân viên chăm sóc khách hàng cần một con số cùng một việc nên làm. Vì vậy mỗi kết quả luôn đi kèm ngữ cảnh để đọc nó: ngưỡng quyết định, chỉ số trên tập test, độ trễ và một lời nhắc khi con số không nên được tin quá mức. Trang gồm ba phần.

- **Phần đầu trang** (Hình 5.2): nhãn chủ đề, tiêu đề chữ chuyển màu, một dòng mô tả hai bài toán và bốn ô số liệu đọc từ `/api/meta`: số mô hình, số phiên AMZN, số người dùng KKBox và độ trễ trung vị của các mô hình PyTorch. Hai nút lớn chuyển giữa hai phân hệ, mỗi phân hệ một màu nhấn (chàm cho giá cổ phiếu, xanh lục cho churn) để người dùng luôn biết mình đang ở đâu.
- **Cột trái**: biểu đồ tương tác (di chuột để xem giá trị từng ngày), dải thống kê nhanh và bảng so sánh bốn kiến trúc.
- **Cột phải** (có định khi cuộn): chọn framework bằng công tắc hai nấc, chọn kiến trúc bằng bốn thẻ mô hình, nút chạy và khung kết quả.



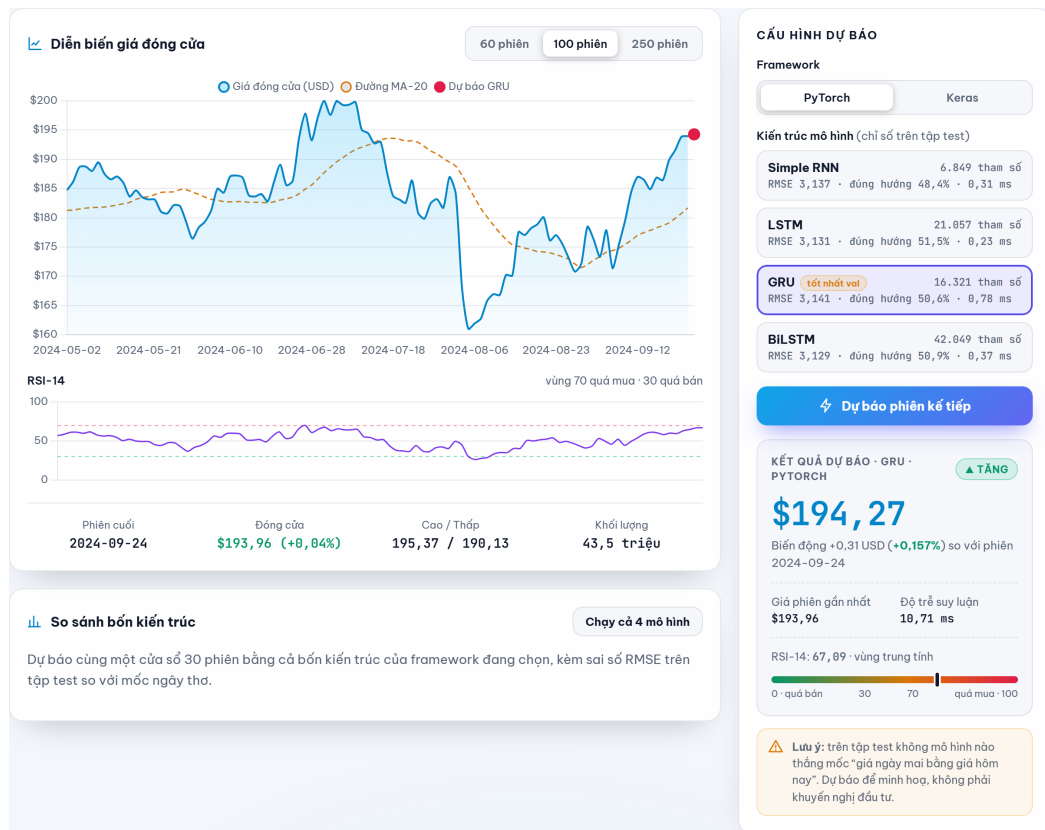
Hình 5.2. Phần đầu trang: bốn ô số liệu và hai nút chuyển phân hệ

Mỗi thẻ mô hình hiện số tham số, hai chỉ số chính trên tập test (RMSE và tỉ lệ đúng hướng cho giá, F1 và ROC-AUC cho churn) và độ trễ đo ở Chương 4; mô hình có val loss nhỏ nhất mang nhãn “tốt nhất val” và được chọn sẵn mỗi khi đổi framework. Người dùng nhờ đó chọn mô hình với đủ thông tin thay vì chọn theo tên. Thẻ lấy số liệu từ hàm `model_card` của máy chủ:

```
1 # app/app.py
2 def model_card(key):
3     dataset = key.split("_")[0]
4     b = BENCH["models"][key]
5     card = {k: round(b[k], 4) for k in CARD_KEYS[dataset]}
6     card.update(params=b["params"], latency_ms=round(b["latency_ms"], 3), size_kb=round(b["size_kb"], 1),
7               best_epoch=b["best_epoch"], val_loss=round(b["best_val_loss"], 4))
8     return card
```

Mã nguồn 32. Thẻ mô hình: chỉ số test, số tham số, độ trễ và kích thước tệp

Các chi tiết nhỏ làm giao diện dễ dùng hơn: nút chạy hiện vòng quay và khoá lại trong lúc chờ API, con số kết quả chạy từ 0 lên giá trị thật, thanh xác suất và kim RSI trượt tới vị trí mới, lỗi từ API hiện thành thông báo nổi ở cuối trang thay vì im lặng. Trang có nền tối mặc định và một nút đổi sang nền sáng ở góc phải (Hình 5.3); lựa chọn được nhớ trong trình duyệt và biểu đồ vẽ lại theo bảng màu mới. Dưới 980 px hai cột xếp chồng, dưới 640 px các ô số liệu chia hai cột và nút phân hệ trải hết chiều ngang, nên trang dùng được trên điện thoại.

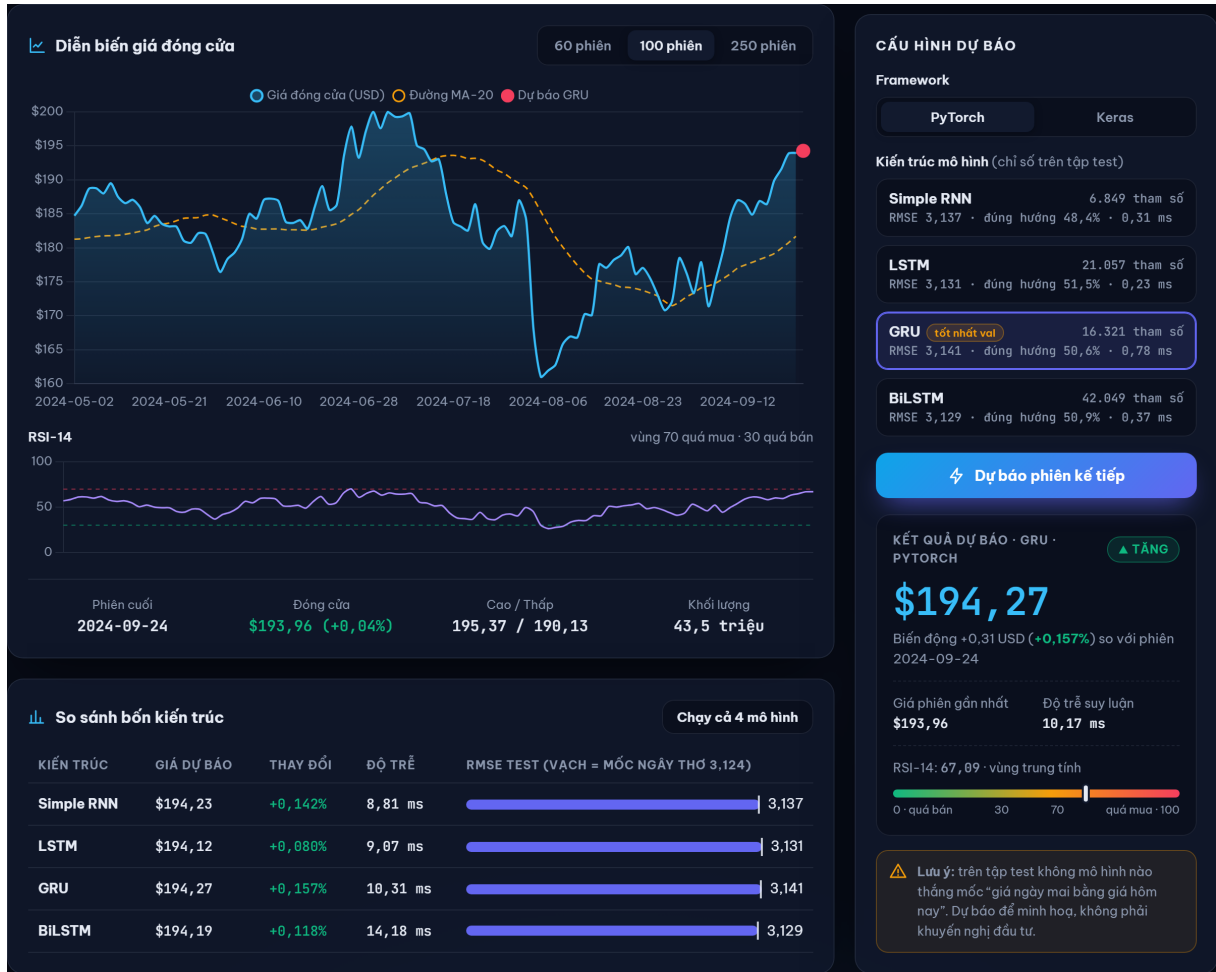


Hình 5.3. Phân hệ 1 ở nền sáng

5.5 Giao diện phân hệ 1: dự báo giá cổ phiếu AMZN

Phân hệ 1 (Hình 5.4) phục vụ bài toán dự báo chuỗi thời gian tài chính.

- **Biểu đồ giá tương tác.** Đường giá đóng cửa có vùng tô chuyển màu và đường MA-20 nét đứt để thấy xu hướng ngắn hạn. Công tắc phía trên đổi khung nhìn 60, 100 hoặc 250 phiên. Sau khi dự báo, một điểm đỏ nổi từ phiên cuối sang “phiên kế tiếp” đặt dự báo ngày trong bối cảnh diễn biến giá.
- **Biểu đồ RSI-14** bên dưới, có hai vạch 70 và 30, cho thấy cổ phiếu đang gần vùng quá mua hay quá bán.
- **Dải thống kê phiên cuối:** ngày, giá đóng cửa và % thay đổi (xanh khi tăng, đỏ khi giảm), giá cao/thấp, khối lượng giao dịch.
- **Khung kết quả:** giá dự báo cỡ lớn, nhãn TĂNG hoặc GIẢM, biến động theo USD và theo %, giá phiên gần nhất, độ trễ suy luận, và một thanh RSI nhiều màu với kim chỉ vị trí hiện tại.
- **Cảnh báo** viền vàng nhắc rằng trên tập test không mô hình nào thắng mốc “giá ngày mai bằng giá hôm nay”.
- **So sánh bốn kiến trúc:** một nút chạy lần lượt cả bốn mô hình trên cùng cửa sổ 30 phiên, rồi xếp giá dự báo, % thay đổi, độ trễ và RMSE test của từng mô hình thành bảng; thanh RMSE có vạch đánh dấu RMSE của mốc ngày thơ nên thấy ngay không thanh nào ngắn hơn vạch.



Hình 5.4. Phân hệ 1: dự báo giá AMZN phiên kế tiếp bằng mô hình PyTorch tốt nhất trên validation, kèm bảng so sánh bốn kiến trúc

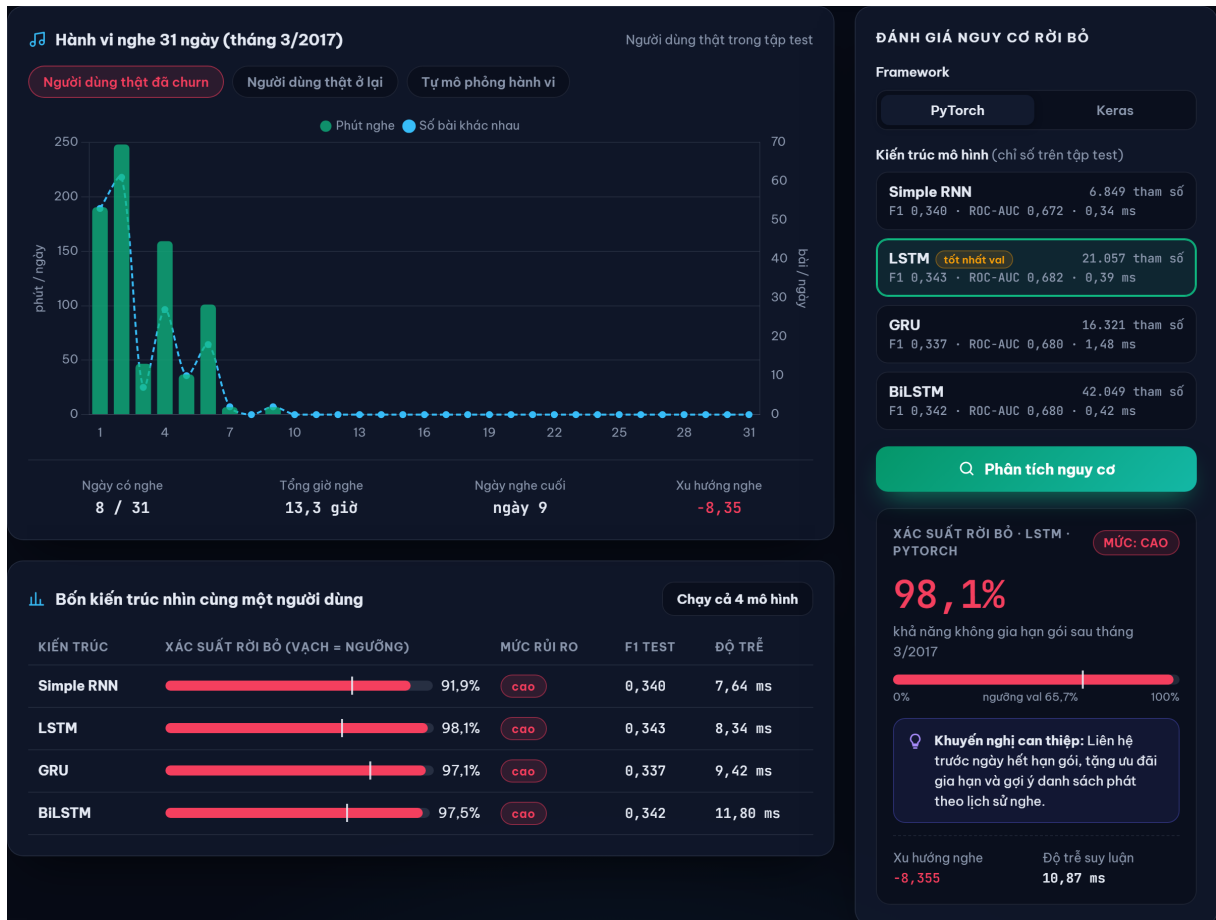
Với dữ liệu kết thúc ngày 2024-09-24, mô hình GRU của PyTorch dự báo phiên kế tiếp đóng cửa ở 194,27 USD, mô hình LSTM của Keras dự báo 194,52 USD, so với giá hiện tại 193,96 USD. Cả hai chỉ lệch vài phần nghìn so với giá hiện tại, và bảng so sánh cho thấy bốn kiến trúc cùng dự báo một mức tăng rất nhỏ, đúng với điều Chương 4 đã phân tích: mô hình dự báo return gần 0.

5.6 Giao diện phân hệ 2: đánh giá nguy cơ rời bỏ KKBox

Phân hệ 2 (Hình 5.5) giúp bộ phận chăm sóc khách hàng quyết định có cần can thiệp để giữ một người dùng hay không.

- **Chọn chuỗi hành vi bằng một chạm:** “Người dùng thật đã churn”, “Người dùng thật ở lại” hoặc “Tự mô phỏng hành vi”.
- **Biểu đồ hai trục** cho 31 ngày tháng 3/2017: cột xanh là số phút nghe mỗi ngày (trục trái), đường nét đứt là số bài khác nhau (trục phải). Hai đại lượng được đổi ngược từ dạng $\log(1+x)$ mà mô hình dùng về đơn vị thật, vì người đọc hiểu “90 phút” dễ hơn “8,6”.
- **Dải thống kê:** số ngày có nghe, tổng giờ nghe, ngày nghe cuối và xu hướng nghe (7 ngày cuối trừ 7 ngày đầu), để người dùng tự kiểm tra điều mô hình vừa kết luận.
- **Khung kết quả:** xác suất rời bỏ cỡ lớn đổi màu theo mức rủi ro (đỏ, vàng, xanh), nhãn mức rủi ro, một thanh xác suất có vạch trắng đánh dấu ngưỡng quyết định của đúng mô hình đó, khuyến nghị can thiệp theo mức rủi ro, xu hướng nghe và độ trễ.

- **Bốn kiến trúc nhìn cùng một người dùng:** xác suất của cả bốn mô hình trên cùng thang, mỗi thanh có vạch ngưỡng riêng, kèm mức rủi ro, F1 test và độ trễ. Bảng tự xoá khi đổi chuỗi hay framework để không bao giờ hiện kết quả của một đầu vào khác.



Hình 5.5. Phân hệ 2: đánh giá nguy cơ rời bỏ của một người dùng thật đã churn trong tập test

Hai người dùng mẫu là người thật trong tập test: một người đã churn mà mô hình tin chắc nhất, và một người ở lại mà mô hình cho xác suất churn thấp nhất. Người churn trong Hình 5.5 nghe gần như mỗi ngày trong 9 ngày đầu tháng rồi ngừng hẳn, dạng cực đoan của quỹ đạo nhóm churn ở Chương 2. LSTM của PyTorch cho xác suất 98,05 % với người này và 23,48 % với người ở lại; GRU của Keras cho lần lượt 98,71 % và 18,32 %.

5.6.1 Trình mô phỏng hành vi

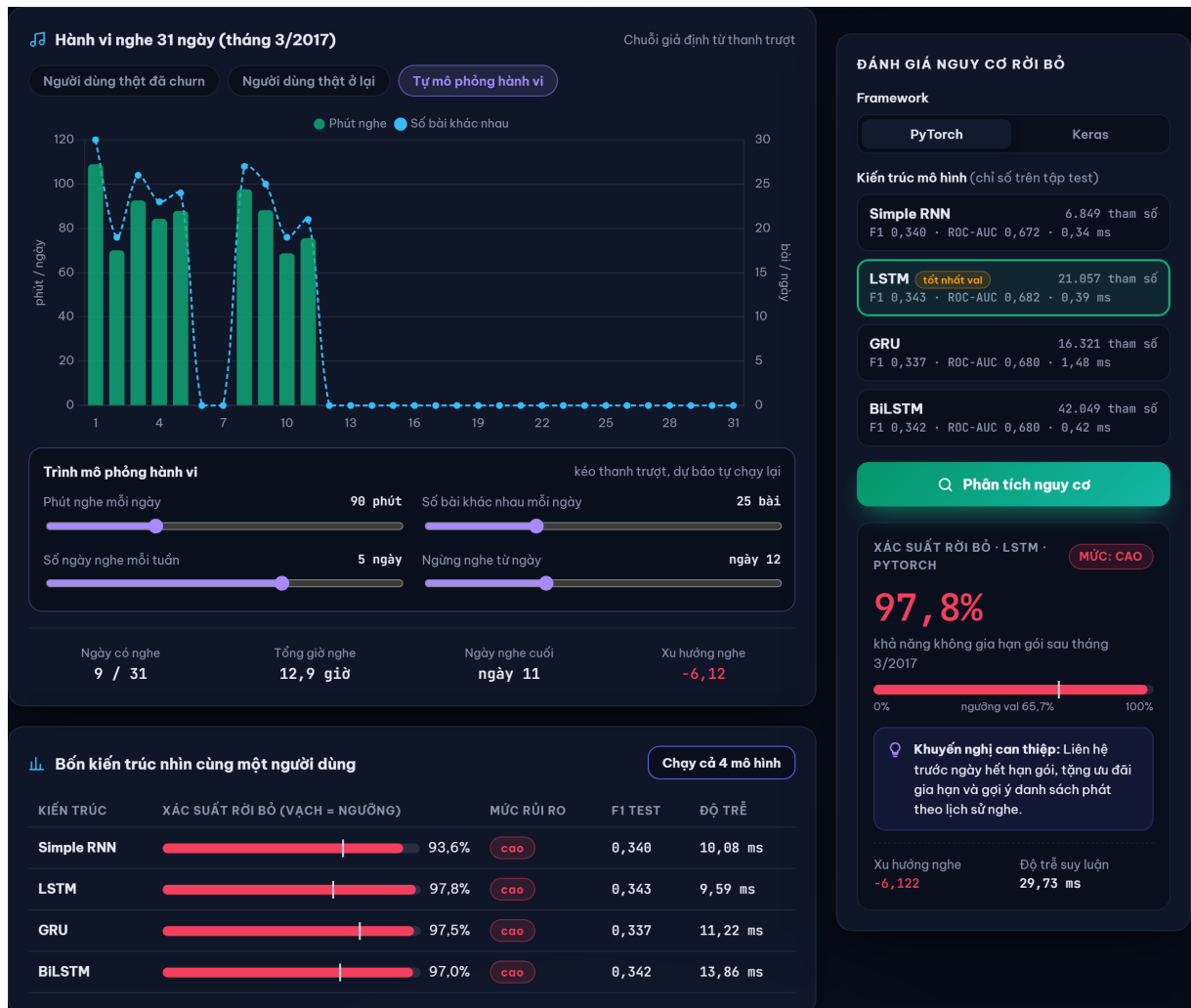
Hai người dùng mẫu chỉ cho biết mô hình phản ứng thế nào với hai trường hợp đã có. Trình mô phỏng cho người dùng tự dựng một tháng nghe nhạc bằng bốn thanh trượt: số phút nghe mỗi ngày, số bài khác nhau mỗi ngày, số ngày nghe mỗi tuần và ngày bắt đầu ngừng nghe. Trình duyệt dựng ma trận 31×8 cùng dạng với dữ liệu huấn luyện (các đếm lấy $\log(1 + x)$, ngày không nghe để 0 và cờ active bằng 0), vẽ lại biểu đồ ngay và gửi chuỗi tới `/api/churn/predict` khi người dùng dừng kéo 0,3 giây. Tỷ lệ giữa các mức nghe (hết bài, bỏ ở 25 %, 50 %...) là giá định, và mỗi ngày có một hệ số dao động cố định để chuỗi trông giống hành vi thật hơn một đường thẳng.

```

1 // app/static/app.js
2 function simulatedSequence() {
3   const minutes = +$("#r-min").value, songs = +$("#r-song").value, perWeek = +$("#r-week").value, stop =
4     +$("#r-stop").value;
5   const seq = [];
6   for (let d = 1; d <= 31; d++) {
7     const row = Array(8).fill(0);
8     if ((d - 1) % 7 < perWeek && d < stop && minutes > 0) {
9       const jitter = 0.75 + 0.5 * Math.abs(Math.sin(d * 12.9898) * 43758.5453 % 1);
10      const plays = songs * 1.3 * jitter;
11      row[F("num_25")] = Math.log1p(Math.round(plays * 0.15));
12      row[F("num_50")] = Math.log1p(Math.round(plays * 0.05));
13      row[F("num_75")] = Math.log1p(Math.round(plays * 0.04));
14      row[F("num_985")] = Math.log1p(Math.round(plays * 0.04));
15      row[F("num_100")] = Math.log1p(Math.round(plays * 0.72));
16      row[F("num_unq")] = Math.log1p(Math.round(songs * jitter));
17      row[F("total_secs")] = Math.log1p(minutes * 60 * jitter);
18      row[F("active")] = 1;
19    }
20    seq.push(row);
21  }
22  return seq;
23 }

```

Mã nguồn 33. Dựng chuỗi 31 ngày giả định từ bốn thanh trượt



Hình 5.6. Trình mô phỏng: nghe 90 phút, 25 bài mỗi ngày, 5 ngày một tuần, ngừng nghe từ ngày 12

Trình mô phỏng cho thấy mô hình nhảy với *thời điểm* ngừng nghe hơn với lượng nghe. Cùng một người nghe 90 phút mỗi ngày, 5 ngày một tuần, nếu nghe đều tới cuối tháng thì LSTM của PyTorch cho xác suất dưới ngưỡng; nếu dừng từ giữa tháng thì xác suất vượt ngưỡng xa và cả bốn kiến trúc đều xếp mức rủi ro cao (Hình 5.6). Điều này khớp với Chương 2: tín hiệu mạnh nhất là hoạt động giảm dần về cuối tháng, và mạng hồi quy đọc được tín hiệu đó vì nó xử lý chuỗi theo thứ tự ngày.

5.7 Kiểm chứng

smoke_test.py dùng test client của Flask, không cần mở cổng mạng. Nó gọi /api/meta (phải đủ 16 thẻ mô hình), lịch sử giá với ba khung 60, 100 và 250 phiên (phải trả đúng số phiên), dự báo giá với cả hai framework và dự báo churn cho hai người dùng mẫu với cả hai framework, tổng cộng 10 lượt. Mỗi phản hồi phải có mã 200 và đủ các khoá cần thiết. Hai đầu vào sai phải bị từ chối: một chuỗi sai shape nhận mã 400, một khung lịch sử ngoài 60/100/250 cho phép nhận mã 400. Kịch bản dừng ngay khi có một điều kiện không thoả, và lượt chạy cuối qua hết.

```

1 # app/smoke_test.py
2 def main():
3     c = app.test_client()
4     meta = check("meta", call(c, "GET", "/api/meta"))
5     if len(meta["cards"]) != 16:
6         raise SystemExit(f"meta: cần 16 thẻ mô hình, có {len(meta['cards'])}")
7     hist = check("history", call(c, "GET", "/api/stock/history"))
8     for n in (60, 250):
9         if len(check("history", call(c, "GET", f"/api/stock/history?n={n}"))["close"]) != n:
10             raise SystemExit(f"history?n={n} trả sai số phiên")
11     stock = {fw: check("stock", call(c, "POST", "/api/stock/predict", {"framework": fw})) for fw in ("torch", "keras")}
12     churn = {f"{p}_{fw}": check("churn", call(c, "POST", "/api/churn/predict", {"preset": p, "framework": fw}))
13              for p in ("churn", "loyal") for fw in ("torch", "keras")}
14     bad = c.post("/api/churn/predict", json={"sequence": [[0] * 8] * 5}).status_code # shape sai phải bị từ chối
15     bad_n = c.get("/api/stock/history?n=7").status_code # dài Lịch sử ngoài 60/100/250
16
17     n = len(hist["close"])
18     save_metrics("app", {"history": {"n": n, "first": hist["dates"][0], "last": hist["dates"][-1],
19                                   "close_last": hist["close"][-1]},
20                        "stock": stock, "churn": {k: {kk: v for kk, v in r.items() if kk != "sequence"}
21                                                  for k, r in churn.items()},
22                        "bad_shape_status": bad, "bad_n_status": bad_n,
23                        "n_calls": 4 + len(stock) + len(churn)})
24     print("4 endpoint trả 200:", 4 + len(stock) + len(churn), "lượt gọi · sai shape ->", bad, "· n sai ->", bad_n)
25     for k, r in stock.items():
26         print(f" stock {k}: {r['model_label']} {r['last_close']} -> {r['pred_close']} USD ({r['latency_ms']} ms)")
27     for k, r in churn.items():
28         print(f" churn {k}: {r['model_label']} p = {r['prob']} · rủi ro {r['risk']} ({r['latency_ms']} ms)")
29     if bad != 400 or bad_n != 400:
30         raise SystemExit("chuỗi sai shape và n sai phải trả 400")

```

Mã nguồn 34. Kiểm chứng bốn endpoint và ghi phản hồi thật cho báo cáo

Ngoài kiểm chứng API, kịch bản screenshots.py trong thư mục app/ mở trang thật bằng Playwright, bấm qua cả hai phân hệ, trình mô phỏng và nền sáng, chụp các hình trong chương này và dừng nếu trang có lỗi JavaScript.

Chạy ứng dụng bằng python app/app.py rồi mở http://localhost:8080. Độ trễ API trả về là thời gian của riêng lời gọi mô hình: 0,3 ms với LSTM của PyTorch và 2,45 ms với GRU của Keras, cùng cỡ với số đo ở Bảng 4.2. Trong ứng dụng, PyTorch chạy 1 luồng còn TensorFlow dùng số luồng mặc định, và mỗi con số chỉ là một lần gọi, nên chúng để tham khảo chứ không thay cho phép đo có kiểm soát của Chương 4.

Chương 6

Kết luận

6.1 Trả lời sáu hạng mục

1. **Cơ sở lý thuyết.** Mỗi khái niệm của mạng hồi quy (tensor chuỗi, Simple RNN, BPTT, clipping, tế bào LSTM và GRU, BiLSTM, recurrent dropout, LayerNorm) được viết thành một hàm NumPy và khớp với PyTorch ở cả 11 phép đối chiếu. Phép đo gradient bằng autograd cho một kết quả đáng nhớ: LSTM chỉ giữ được gradient khi công quên mở. Với bias công quên bằng 0, gradient ở trễ 20 gần như bằng của Simple RNN.
2. **Hai tập dữ liệu thực tế.** AMZN có 6.885 phiên, chia theo thời gian, dự báo return thay vì mức giá vì giá không dừng (ADF $p = 0,9963$). KKBox có 100.000 người dùng thật lấy từ 18.396.362 dòng log, đọc theo khối không cần nạp trọn vào RAM. Trên cả hai tập, mọi thống kê chuẩn hoá chỉ tính trên train.
3. **PyTorch.** Bốn mô hình dùng chung một lớp và một vòng lặp huấn luyện, huấn luyện trên GPU sau khi đo được GPU nhanh hơn CPU. Trên KKBox, LSTM tốt nhất với F1 34,26 % và ROC-AUC 0,6819; về ROC-AUC, ba mô hình có công đều hơn Simple RNN.
4. **Keras và đối chuẩn.** Cùng kiến trúc cho chất lượng gần như bằng nhau ở hai framework. PyTorch nhanh hơn với từng mẫu một (trung vị 0,383 so với 1,44 ms trên một luồng CPU), còn với lô lớn thì khoảng cách hẹp lại và Keras thắng ở Simple RNN. Tập .keras nặng hơn vì chứa cả trạng thái optimizer.
5. **Web App.** Flask phục vụ cả 16 mô hình qua REST API; giao diện hai phân hệ nền tối/sáng có biểu đồ tương tác, thẻ mô hình kèm chỉ số test, bảng so sánh bốn kiến trúc và trình mô phỏng hành vi nghe nhạc. Toàn bộ tham số tiền xử lý đọc từ metadata.json nên ứng dụng chạy được mà không cần dữ liệu gốc. Kịch bản kiểm chứng qua 10 lượt gọi.
6. **Kết luận về mô hình.** Mạng hồi quy học được tín hiệu thật khi tín hiệu nằm trong chuỗi (xu hướng nghe nhạc giảm dần của người sắp rời bỏ). Nó không tạo ra được tín hiệu khi chuỗi không chứa (return cổ phiếu ngày mai): không mô hình nào trong 16 mô hình thắng mốc “giá ngày mai bằng giá hôm nay”.

6.2 Hạn chế

- Mỗi mô hình chỉ chạy một seed. Chênh lệch giữa các mô hình trên KKBox (dưới hai điểm F1) và giữa hai framework đều nhỏ hơn mức dao động thường gặp giữa các lần khởi tạo, nên bài không xếp hạng chúng.
- KKBox chỉ dùng mẫu 100.000 người và 31 ngày log. Bỏ qua lịch sử giao dịch và thông tin thành viên nên ROC-AUC dừng ở khoảng 0,68, thấp hơn các lời giải đầy đủ của cuộc thi.
- AMZN chỉ dùng đặc trưng kỹ thuật tính từ chính chuỗi giá. Các sự kiện gây biến động lớn nhất (báo cáo quý, tin vĩ mô) không có trong dữ liệu.
- Keras chạy trên CPU vì TensorFlow không hỗ trợ GPU trên Windows, nên thời gian huấn luyện của hai framework không so trực tiếp được; bài chỉ so tốc độ suy luận trên cùng CPU.

6.3 Hướng phát triển

- **Thêm dữ liệu cho bài churn.** Kết quả của Chương 2 cho thấy tín hiệu nằm ở xu hướng hành vi, và phần còn thiếu nằm ngoài log nghe nhạc. Bước có lợi nhất là ghép chuỗi giao dịch (transactions_v2) và thông tin thành viên của cùng cuộc thi vào mô hình, dưới dạng đặc trưng tĩnh nối vào h_T .
- **Chú ý theo thời gian.** Cơ chế attention [21] gán trọng số cho từng bước thay vì chỉ dựa vào h_T , và cho biết ngày nào quyết định dự báo, điều hữu ích cho bộ phận chăm sóc khách hàng.

- **Mô hình chuỗi thời gian hiện đại.** Temporal Fusion Transformer [22], PatchTST [23] và các mô hình không gian trạng thái như Mamba [24] xử lý chuỗi dài hơn nhiều so với LSTM.
- **Đổi mục tiêu cho bài giá.** Với return gần như không dự báo được, câu hỏi hữu ích hơn là dự báo *biến động* (vốn có tự tương quan mạnh, xem hiệu ứng ARCH ở Chương 2) hoặc dự báo xác suất tăng/giảm, rồi so với mốc ngày thơ tương ứng.
- **Đánh giá chắc chắn hơn.** Chạy nhiều seed và kiểm định thống kê để phân biệt được chênh lệch nhỏ giữa các mô hình; với bài giá, dùng kiểm định trượt nhiều giai đoạn (walk-forward) thay cho một lần chia duy nhất.

Tài liệu tham khảo

- [1] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211.
- [2] Paul J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.
- [3] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. “Learning Long-Term Dependencies with Gradient Descent is Difficult”. In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.
- [4] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. In: *Proceedings of the 30th International Conference on Machine Learning*. 2013, pp. 1310–1318.
- [5] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. “An Empirical Exploration of Recurrent Network Architectures”. In: *Proceedings of the 32nd International Conference on Machine Learning*. 2015, pp. 2342–2350.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [7] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *Proceedings of EMNLP 2014*. 2014, pp. 1724–1734.
- [8] Mike Schuster and Kuldip K. Paliwal. “Bidirectional Recurrent Neural Networks”. In: *IEEE Transactions on Signal Processing* 45.11 (1997), pp. 2673–2681.
- [9] Yarın Gal and Zoubin Ghahramani. “A Theoretically Grounded Application of Dropout in Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems* 29. 2016, pp. 1019–1027.
- [10] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450* (2016).
- [11] Yahoo Finance. *Amazon.com, Inc. (AMZN) Stock Historical Prices & Data*. Tài bằng thư viện yfinance, dữ liệu ngày 15/05/1997 – 24/09/2024, truy cập 2026-09-28. 2024. URL: <https://finance.yahoo.com/quote/AMZN/history/>.
- [12] J. Welles Wilder. *New Concepts in Technical Trading Systems*. Trend Research, 1978.
- [13] David A. Dickey and Wayne A. Fuller. “Distribution of the Estimators for Autoregressive Time Series with a Unit Root”. In: *Journal of the American Statistical Association* 74.366 (1979), pp. 427–431.
- [14] Eugene F. Fama. “Efficient Capital Markets: A Review of Theory and Empirical Work”. In: *The Journal of Finance* 25.2 (1970), pp. 383–417.
- [15] KKBOX and WSDM 2018. *WSDM – KKBox’s Churn Prediction Challenge*. Kaggle. Tập train_v2.csv và user_logs_v2.csv, truy cập 2026-09-28. 2017. URL: <https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge/data>.
- [16] Bernard L. Welch. “The Generalization of “Student’s” Problem when Several Different Population Variances are Involved”. In: *Biometrika* 34.1/2 (1947), pp. 28–35.
- [17] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems* 32. 2019, pp. 8024–8035.
- [18] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. 2019.
- [19] François Chollet et al. *Keras*. <https://keras.io>. 2015.

- [20] Martín Abadi, Paul Barham, Jianmin Chen, et al. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation*. 2016, pp. 265–283.
- [21] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30. 2017, pp. 5998–6008.
- [22] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. “Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting”. In: *International Journal of Forecasting* 37.4 (2021), pp. 1748–1764.
- [23] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers”. In: *International Conference on Learning Representations*. 2023.
- [24] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *arXiv preprint arXiv:2312.00752* (2023).

Phụ lục A

Tái lập kết quả

A.1 Môi trường

Windows 11, Python 3.13, GPU NVIDIA GeForce RTX 4060 Laptop cho PyTorch (CUDA 12.6), CPU cho TensorFlow. PyTorch 2.13.0+cu126, Keras 3.15.1 với TensorFlow 2.21. Mọi thư viện cài trong môi trường ảo riêng của bài, liệt kê trong requirements.txt.

A.2 Các bước

1. Tạo môi trường và cài thư viện:

```
cd "src/Assignment 06"
python -m venv .venv
.venv\Scripts\activate
pip install torch==2.13.0 --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
python -m ipykernel install --user --name a06
```

2. Tải train_v2.csv.7z và user_logs_v2.csv.7z của cuộc thi KKBBox trên Kaggle (cần đăng nhập và chấp nhận luật cuộc thi) vào data/kkbox/, giải nén. AMZN.csv đã có sẵn trong kho.
3. Chạy các notebook theo thứ tự số. Notebook 03, 04 (GPU) và 05 (CPU) chạy song song được. Trước khi chạy, đặt biến môi trường OPENBLAS_NUM_THREADS bằng 8: trên máy 32 luồng, OpenBLAS mặc định cấp bộ đệm cho 32 luồng và có lúc làm kernel chết vì không cấp phát được bộ nhớ.

```
cd notebooks
jupyter nbconvert --to notebook --execute --inplace ^
--ExecutePreprocessor.kernel_name=a06 --ExecutePreprocessor.timeout=-1 00_*.ipynb
```

4. Kiểm chứng Web App: python app/smoke_test.py; chạy thật: python app/app.py.
5. Dựng báo cáo: python "Report/Assignment 06/build.py" (cần LuaLaTeX, latexmk, biber và gói minted).

Lần chạy đầu của notebook 02 gom log KKBBox thành tệp kkbox_seq.npz; các lần sau đọc thẳng tệp này.

```
1 # a06/data_churn.py
2 def build_cache():
3     """Một lần: Lấy mẫu, gom chuỗi, chia tập, Lưu .npz. Trả về dict thông tin để ghi metrics."""
4     sample, info = sample_users()
5     X, info2 = build_sequences(sample)
6     y = sample.is_churn.to_numpy(np.int64)
7     split = stratified_split(y)
8     np.savez_compressed(CACHE, X=X, y=y, msno=sample.msno.to_numpy(str),
9                        **{f"idx_{k}": v for k, v in split.items()})
10    info = {**info, **info2, **{f"n_{k}": len(v) for k, v in split.items()}}
11    CACHE_INFO.write_text(json.dumps(info, indent=2), encoding="utf-8")
12    return info
```

Mã nguồn 35. Gom dữ liệu KKBBox một lần và lưu lại